Отчёт по лабораторной работе №2

Модели SIR и Лотки-Вольтерры

Селиванов Вячеслав Алексеевич

Содержание

1 Цель работы **6**

2 Задание **7**

3 Теоретическое введение **8**

4 Выполнение лабораторной работы **9**

5 Работа с трехпараметрической моделью SIR **15**

5.1 Ключевая идея 15
5.2 Декомпозиция заразности 15
5.3 Математическое описание 16
5.4 Итоговая система уравнений: 16
5.5 Базовое репродуктивное число 17
5.6 Параметры модели 17
5.7 Расчет базового репродуктивного числа 18

6 Создание и решение задачи **19**

6.1 Подготовка данных в DataFrame 20
6.2 Вывод параметров модели 21
6.3 1. ОСНОВНОЙ ГРАФИК: динамика всех трех групп 22
6.4 График только инфицированных (I) 24
6.5 Отметка пика эпидемии 25
6.6 График в логарифмическом масштабе (для анализа экспоненциального роста) 26
6.7 График долей населения (в процентах) 27
6.8 Горизонтальная линия для порога коллективного иммунитета 28
6.9 Фазовый портрет (I vs S) 29
6.10 Добавление стрелок направления 30
6.11 График R_e - эффективного репродуктивного числа 30
6.12 Горизонтальная линия на уровне 1 31
6.13 Отметка момента, когда R_e становится < 1 32
6.14 Компактный график всех кривых в одной панели 33
6.15 Сохранение графиков 36
6.16 Бенчмарк для оценки производительности 37
6.17 Дополнительный анализ 37

7 Описание модели Лотки-Вольтерры **40**

8 Параметры модели и начальные условия **42**

9 Подготовка данных **46**

10 Рассчет производных для анализа **48**

11 Вывод информации о модели **50**

12 Стационарные точки (нулевые изоклины) **52**

13 Построение графиков **53**
13.1 График 1: Динамика популяций во времени 53
13.2 График 2: Фазовый портрет (хищники vs жертвы) 55
13.3 График 3: Производные (скорости изменения) 58
13.4 График 4: Относительные изменения (в %) 58
13.5 График 5: Спектральный анализ (быстрое преобразование Фурье) . . 60
13.6 График 6: Компактная панель всех графиков 63

14 Анализ результатов **66**

15 Сохранение графиков **68**

16 Дополнительный анализ: чувствительность к параметрам **70**
16.1 Анализ чувствительности к ключевым параметрам 71

17 Выводы **73**

Список литературы **74**

Список иллюстраций

4.1 Инициализация проекта 9
4.2 Добавление пакетов 9
4.3 Результаты выполнения скрипта для модели SIR 9
4.4 Создание производных форматов 10
4.5 Файл .ipynb 10
4.6 Результаты выполнения скрипта для модели Лотки-Вольтерры 11
4.7 Создание производных форматов 11
4.8 Файл .ipynb 12
4.9 Перебор параметров альфа и гамма 12
4.10 Создание производных форматов 13
4.11 Выполненим файл .ipynb 13

Список таблиц

1 Цель работы

Изучить и проанализировать модели SIR и Лотки-Вольтерры

2 Задание

Создать скрипты, анализирующие и визуализирующие две основные модели: SIR и модель Лотки-Вольтерры

3 Теоретическое введение

SIR

Модель SIR делит всю популяцию на три взаимосвязанные группы (компартменты), что отражено в её названии: — □ — Susceptible (Восприимчивые): люди, которые не болели, не имеют иммунитета и могут заразиться. — □ — Infectious (Инфицированные/Заразные): люди, которые в данный момент больны и могут передавать инфекцию. — □ — Recovered (Выздоровевшие/Удаленные): люди, которые переболели и приобрели иммунитет (или умерли). Они больше не участвуют в передаче

Основная цель модели: не предсказать судьбу конкретного человека, а понять общую динамику эпидемии — будет ли она разрастаться, как быстро, сколько людей в итоге переболеет, как влияют карантинные меры 1. Уравнение для восприимчивых (□): □□ □□ = –□□□/N 2. Уравнение для заразных (□): □□ □□ = □□□/□ – □I

3. Уравнение для выздоровевших (□): □□ □□ = □I

Модель Лотки-Вольтерры

Модель Лотки-Вольтерры — это фундаментальная математическая модель в экологии, описывающая динамику взаимодействия двух видов: хищников и жертв Уравнение для популяции жертв (x) □□ □□ = □□ – □□y Уравнение для популяции хищников (y) □□ □□ = □□□ – □y

4 Выполнение лабораторной работы

Создадим проект для лабораторной работы (рис. 4.1).

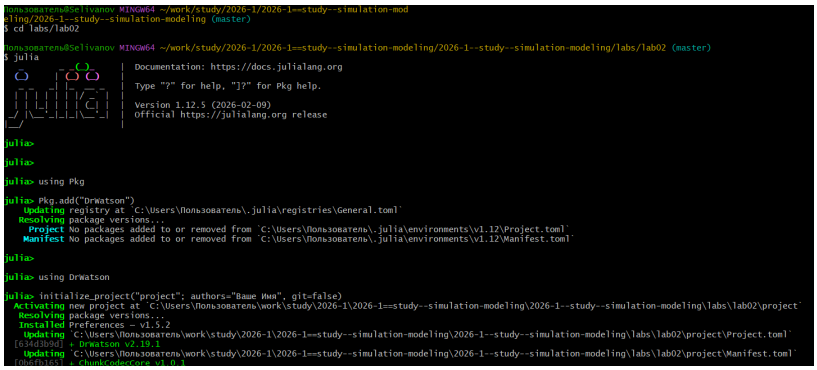


Рисунок 4.1: Инициализация проекта

Добавляем необходимые пакеты (рис. 4.2).

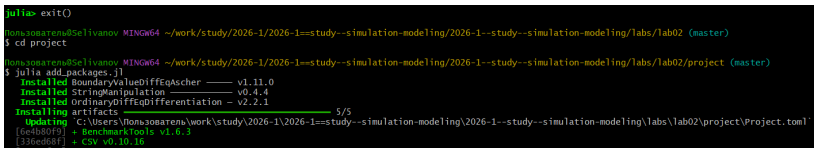


Рисунок 4.2: Добавление пакетов

Создадим скрипт для работы с моделью SIR и запустим его (рис. 4.3).

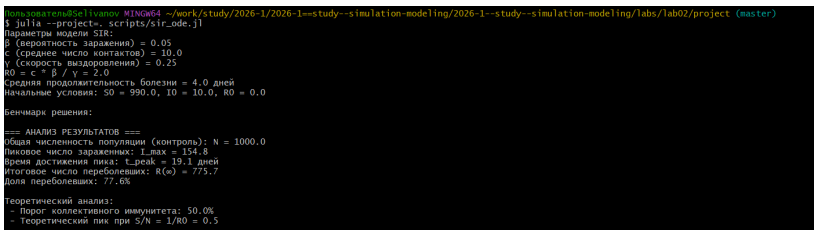


Рисунок 4.3: Результаты выполнения скрипта для модели SIR

Создадим производные форматы (рис. 4.4).

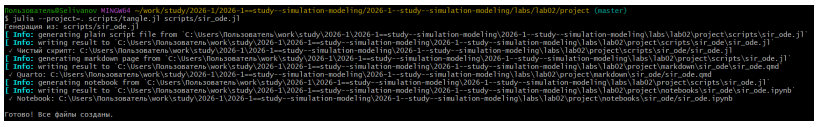


Рисунок 4.4: Создание производных форматов

Проверим файл для Jupyter (рис. 4.5).

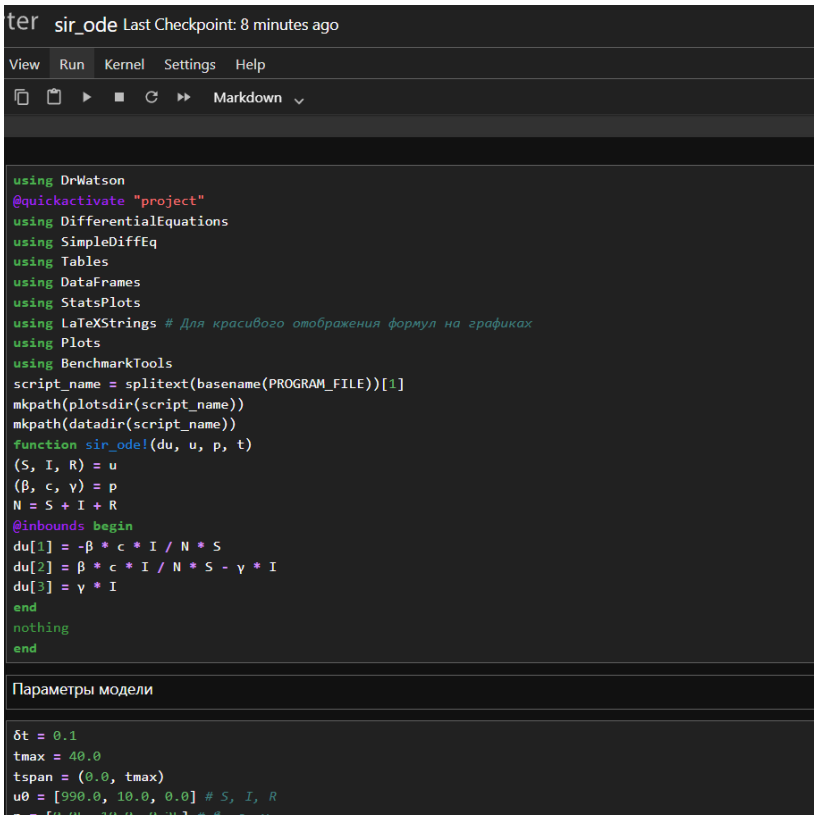


Рисунок 4.5: Файл .ipynb

Создадим скрипт для работы с моделью Лотки-Вольтерры и запустим его (рис. 4.6).

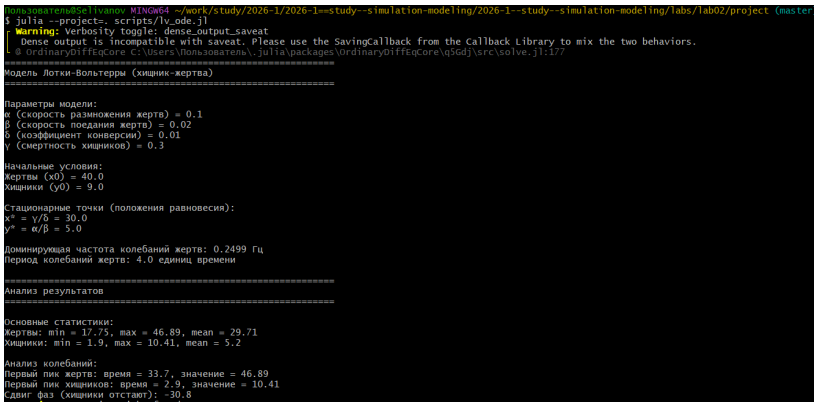


Рисунок 4.6: Результаты выполнения скрипта для модели Лотки-Вольтерры

Создадим производные форматы (рис. 4.7).

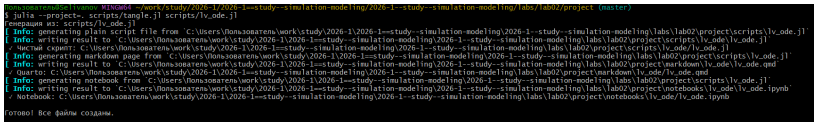


Рисунок 4.7: Создание производных форматов

Проверим файл для Jupyter для модели Лотки-Вольтерры (рис. 4.8).

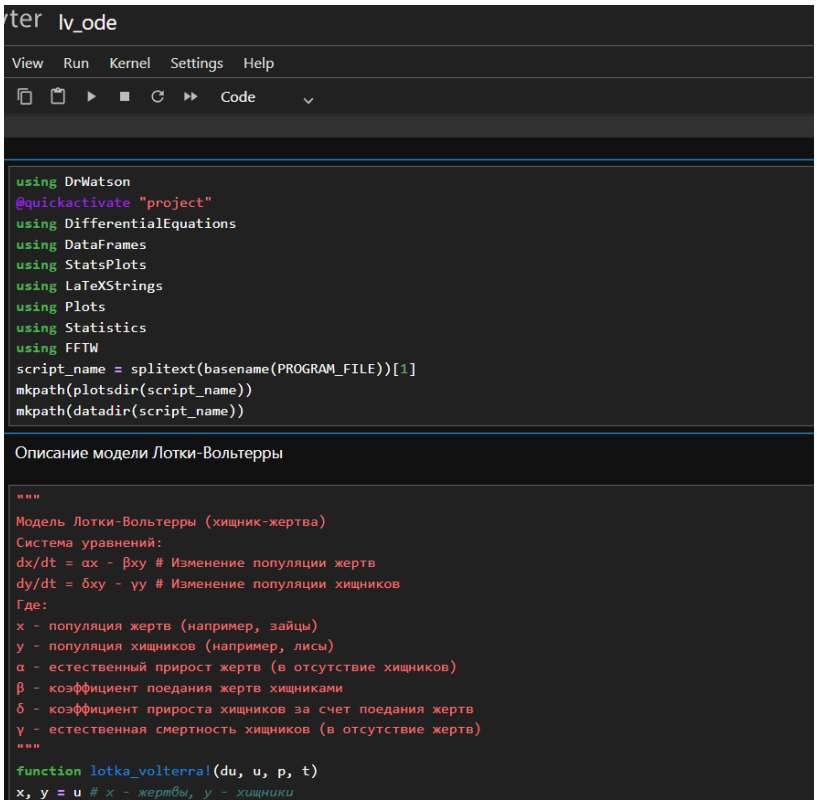


Рисунок 4.8: Файл .ipynb

Изменим скрипт, чтобы он перебирал параметры альфа и гамма и проанализируем чувствительность к этим параметрам (рис. 4.9).

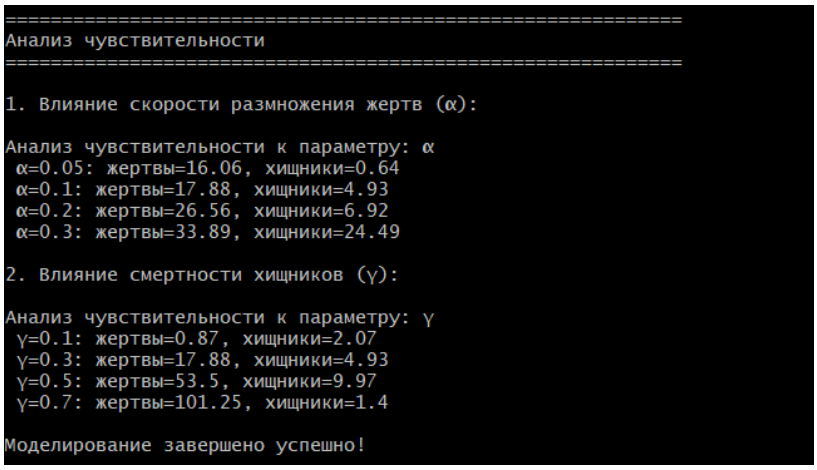


Рисунок 4.9: Перебор параметров альфа и гамма

Создадим производные форматы для модели с перебором параметров (рис. 4.10).

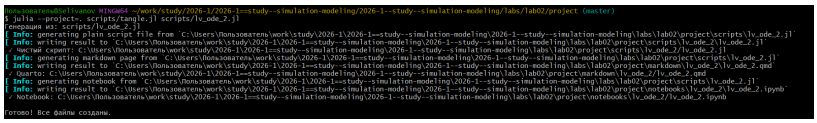


Рисунок 4.10: Создание производных форматов

Выполненим файл .ipynb в Jupyter (рис. 4.11).

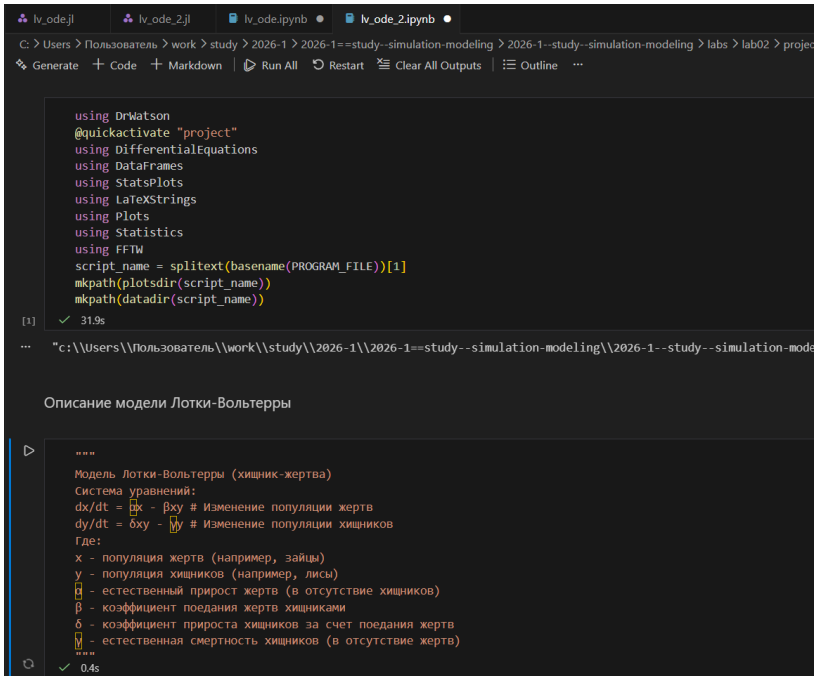


Рисунок 4.11: Выполненим файл .ipynb

```julia
using DrWatson
@quickactivate "project"

using DifferentialEquations
using SimpleDiffEq
using Tables
using DataFrames
using StatsPlots
using LaTeXStrings  # [illegible]
```

```julia
using Plots
using BenchmarkTools
default(fmt = :png)
gr(fmt = :png)
```

```
┌ Warning: DrWatson could not find find a project file by recursively c
⋮ (given dir: C:\Users\12232)
└ @ DrWatson C:\Users\12232\.julia\packages\DrWatson\2QF5p\src\proje
```

```
Plots.GRBackend()
```

Установка пакетов

```julia
script_name = splitext(basename(PROGRAM_FILE))[1]
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))
```

```
"C:\\Users\\12232\\Documents\\GitHub\\2026-1--study--
simulation-modeling\\labs\\lab02\\project\\data\\"
```

5 Работа с трехпараметрической моделью SIR

5.1 Ключевая идея

Модель SIR делит всю популяцию на три взаимосвязанные группы (компартменты), что отражено в её названии:

S — Susceptible (Восприимчивые): люди, которые не болели, не имеют иммунитета и могут заразиться. I — Infectious (Инфицированные/Заразные): люди, которые в данный момент больны и могут передавать инфекцию. R — Recovered (Выздоровевшие/Удаленные): люди, которые переболели и приобрели иммунитет (или умерли). Они больше не участвуют в процессе передачи.

Основная цель модели: не предсказать судьбу конкретного человека, а понять общую динамику эпидемии — будет ли она разрастаться, как быстро, сколько людей в итоге переболеет, как влияют карантинные меры.

5.2 Декомпозиция заразности

В классической двухпараметрической модели (β, γ) параметр β (коэффициент заражения) является составным. Он скрывает в себе два процесса:

- Контакт между людьми (поведенческий, управляемый фактор).
- Передачу инфекции при контакте (биологический фактор).

Трёхпараметрическая модель делает это разделение явным:

c — среднее число контактов (достаточно тесных для передачи инфекции) одного человека в единицу времени. β — вероятность передачи инфекции при одном контакте между заразным и восприимчивым (безразмерная величина, от 0 до 1). γ — скорость выздоровления (доля инфицированных, выздоравливающих в единицу времени). Как и раньше, $1/\gamma$ — средняя продолжительность заразного периода

Итоговый параметр силы заражения теперь выражается как произведение: $c * \beta$

5.3 Математическое описание

Поток новых заражений (dS/dt) теперь формируется так:

У одного заразного (I) в день происходит контактов. Доля восприимчивых среди всех контактов равна S/N. Следовательно, число релевантных контактов с восприимчивыми для всех заразных равно $cI(S/N)$. Но не каждый такой контакт приводит к заражению. Вероятность этого события — β.

5.4 Итоговая система уравнений:

5.4.1 Уравнение для восприимчивых (S):

$dS/dt = -(\beta c)IS/N$

5.4.2 Уравнение для заразных (I):

$dI/dt = (\beta c)IS/N - \gamma I$

5.4.3 Уравнение для выздоровевших (R):

$dR/dt = \gamma I$

5.5 Базовое репродуктивное число

Формула для R_0: $R_0 = (c\beta)/\gamma$

Смысл: $c\beta$ — это среднее число людей, которых заражает один больной за один день, если все вокруг восприимчивы. $1/\gamma$ — это количество дней, которые человек в среднем заразен.

R_0 = (число заражений в день) * (число заразных дней)

```
function sir_ode!(du, u, p, t)
    (S, I, R) = u
    (□, c, □) = p
    N = S + I + R
    @inbounds begin
        du[1] = -□ * c * I / N * S
        du[2] = □ * c * I / N * S - □ * I
        du[3] = □ * I
    end
    nothing
end
```

```
sir_ode! (generic function with 1 method)
```

5.6 Параметры модели

```
□t = 0.1
tmax = 40.0
tspan = (0.0, tmax)
u0 = [990.0, 10.0, 0.0]  # S, I, R
p = [0.05, 10.0, 0.25]   # □, c, □
```

```
3-element Vector{Float64}:
  0.05
 10.0
  0.25
```

5.7 Расчет базового репродуктивного числа

```
R0 = (p[2] * p[1]) / p[3]  # R₀ = (c * □) / □
```

```
2.0
```

6 Создание и решение задачи

```
prob_ode = ODEProblem(sir_ode!, u0, tspan, p)
sol_ode = solve(prob_ode, dt = Δt)
```

```
retcode: Success
Interpolation: 3rd order Hermite
t: 15-element Vector{Float64}:
  0.0
  0.1
  0.5331242537191838
  1.3919008491040001
  2.6102120755226386
  4.1811820758323055
  6.180123810566044
  8.664951930903612
 11.691425331813486
 15.279370667110317
 19.082963490744028
 23.423425193442526
 28.42949373098617
 33.298779173639595
 40.0
```

```
u: 15-element Vector{Vector{Float64}}:
 [990.0, 10.0, 0.0]
 [989.4990153200853, 10.247898072276143, 0.25308660763853125]
 [987.1852964117763, 11.391111426852984, 1.4235921613706284]
 [981.8312125227972, 14.026022042698546, 4.14276543450428]
 [972.1392310589528, 18.757815561654187, 9.102953379393007]
 [954.9905500491583, 27.007702518267607, 18.00174743257403]
 [923.0706717929863, 41.92976779832721, 34.999560408686385]
 [862.6705810342164, 68.49345419063417, 68.83596477514932]
 [754.3022993851442, 109.74186185409279, 135.9558387607629]
 [595.1014268847736, 150.41363264480682, 254.48494047041947]
 [442.0141179101299, 154.80694988491865, 403.1789322049513]
 [326.8951167493959, 119.06860394684855, 554.0362793037555]
 [258.5085242188966, 70.11052194753069, 671.3809538335727]
 [227.58338337085726, 37.328570631579986, 735.0880459975627]
 [209.84121402014364, 14.47364496645583, 775.6851410134005]
```

6.1 Подготовка данных в DataFrame

```
df_ode = DataFrame(Tables.table(sol_ode'))
rename!(df_ode, ["S", "I", "R"])
df_ode[!, :t] = sol_ode.t
df_ode[!, :N] = df_ode.S + df_ode.I + df_ode.R  # □□□□□ □□□□□□□□□□□ □□□□
```

```
15-element Vector{Float64}:
 1000.0
 1000.0
  999.9999999999999
```

```
1000.0
1000.0
 999.9999999999999
1000.0
 999.9999999999999
1000.0
 999.9999999999999
 999.9999999999998
1000.0
1000.0
1000.0
1000.0
```

6.2 Вывод параметров модели

```julia
println("□□□□□□□□□ □□□□□□ SIR:")
println("□ (□□□□□□□□□□□ □□□□□□□□□) = ", p[1])
println("c (□□□□□□□ □□□□□ □□□□□□□□□) = ", p[2])
println("□ (□□□□□□□□ □□□□□□□□□□□□□) = ", p[3])
println("R₀ = c * □ / □ = ", round(R0, digits=3))
println("□□□□□□□ □□□□□□□□□□□□□□□□□ □□□□□□□ = ", round(1/p[3], digits=2), "
println("□□□□□□□□□ □□□□□□□□: S₀ = ", u0[1], ", I₀ = ", u0[2], ", R₀ = ",
```

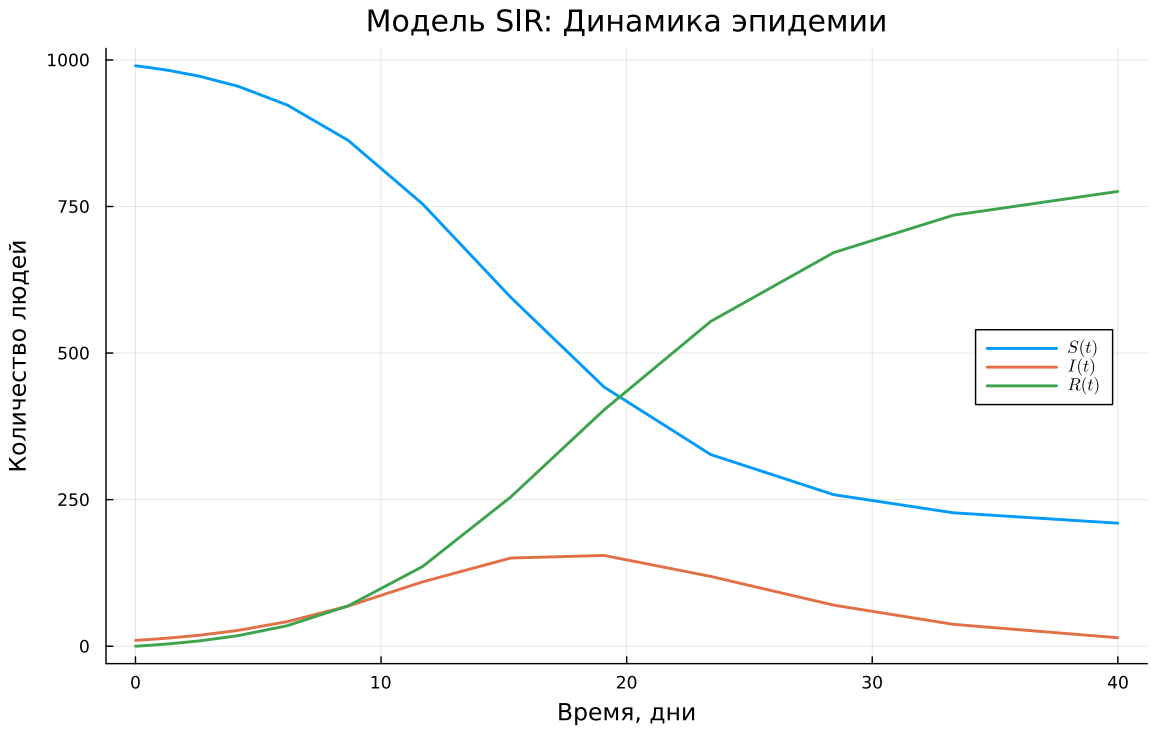
```
□□□□□□□□□ □□□□□□ SIR:
□ (□□□□□□□□□□□ □□□□□□□□□) = 0.05
c (□□□□□□□ □□□□□ □□□□□□□□□) = 10.0
□ (□□□□□□□□ □□□□□□□□□□□□□) = 0.25
R₀ = c * □ / □ = 2.0
```

[illegible] [illegible] [illegible] = 4.0 [illegible]

[illegible]: S_0 = 990.0, I_0 = 10.0, R_0 = 0.0

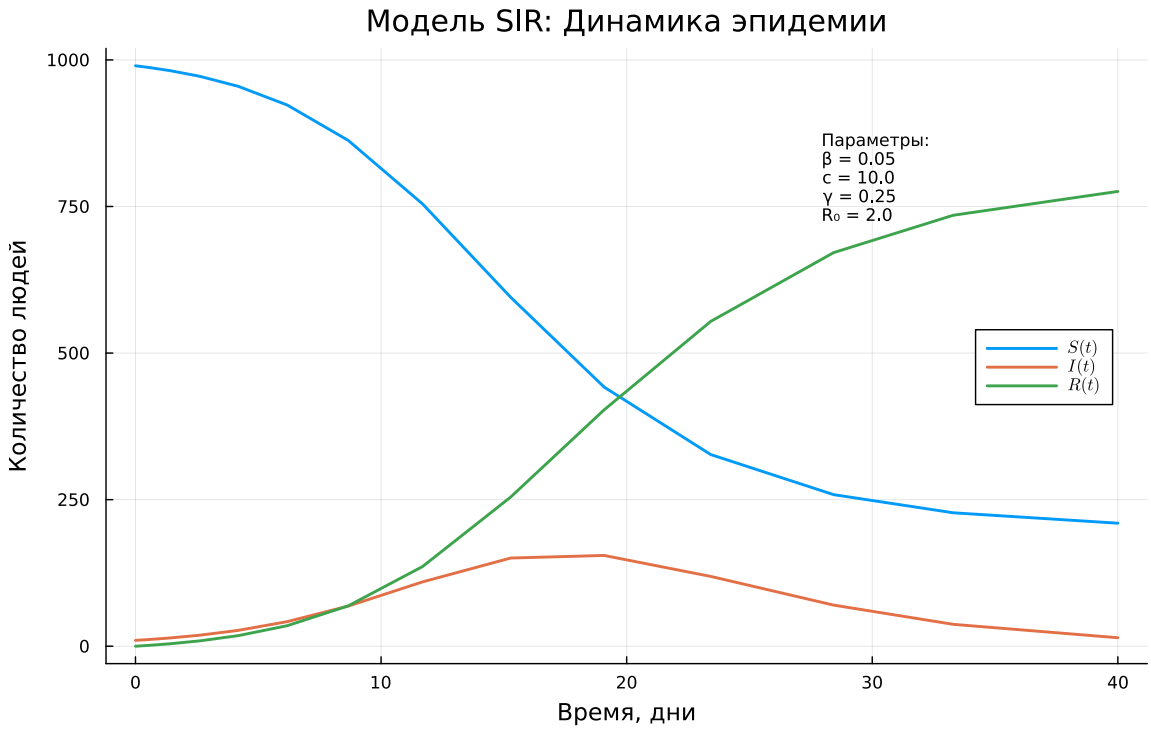
6.3 1. ОСНОВНОЙ ГРАФИК: динамика всех трех групп

```
plt1 = @df df_ode plot(:t,
    [:S :I :R],
    label=[L"S(t)" L"I(t)" L"R(t)"],
    xlabel="[illegible], [illegible]",
    ylabel="[illegible] [illegible]",
    title="[illegible] SIR: [illegible] [illegible]",
    linewidth=2,
    legend=:right,
    grid=true,
    size=(800, 500))
```



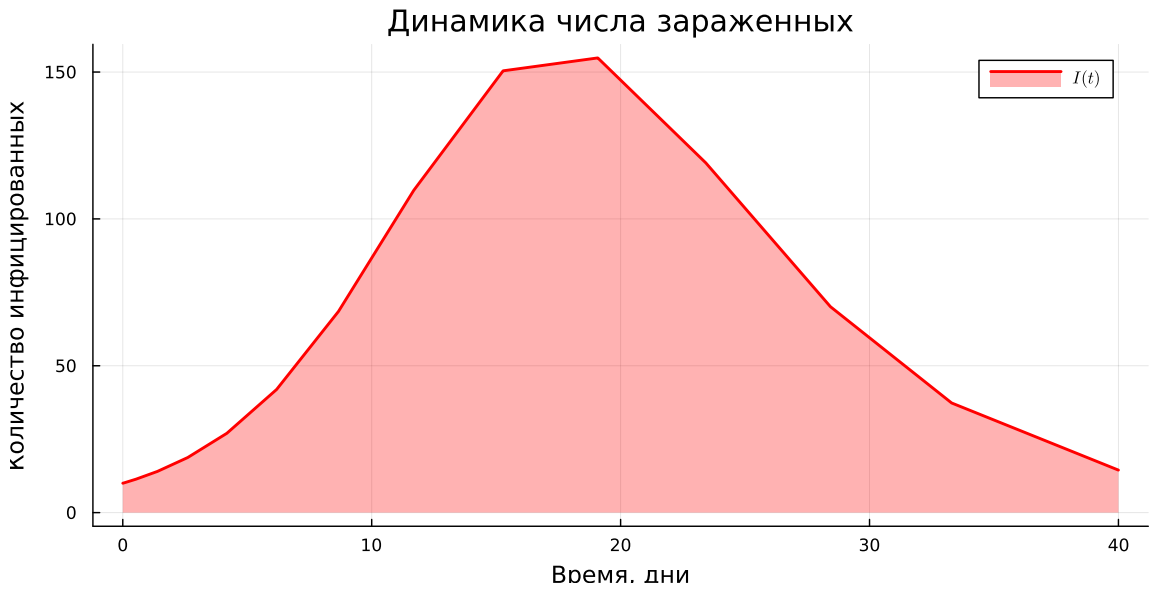
6.3.1 Добавление аннотаций с параметрами

```
annotate!(plt1, maximum(df_ode.t) * 0.7, maximum(df_ode.N) * 0.8,
    text("⊠⊠⊠⊠⊠⊠⊠⊠⊠:\n⊠ = $(p[1])\nc = $(p[2])\n⊠ = $(p[3])\nR₀ = $(ro
    8, :left))
```



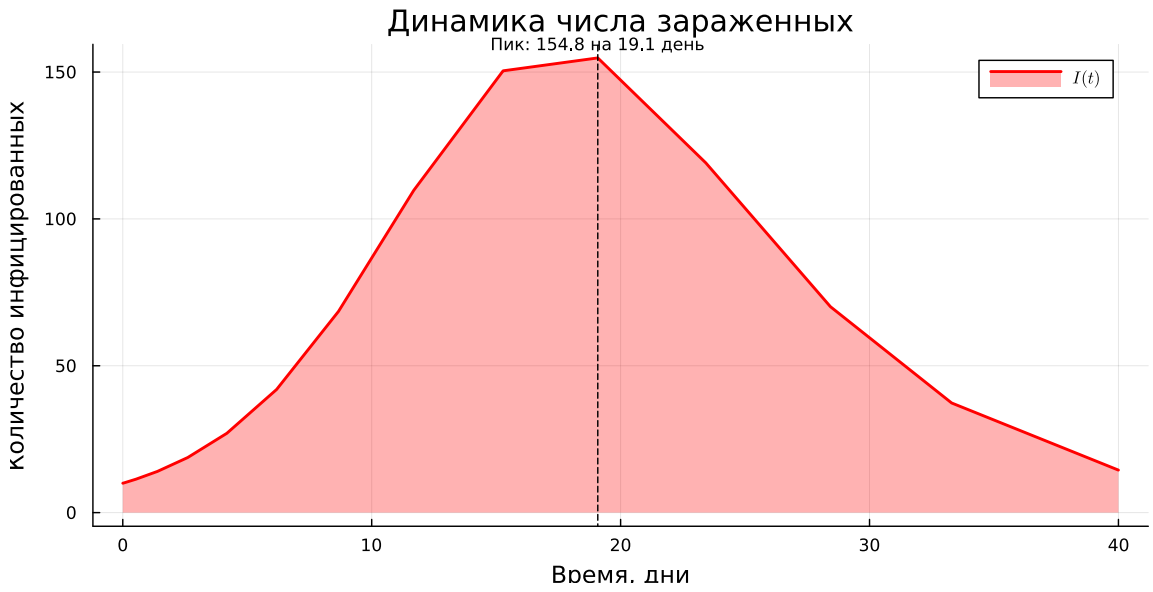
6.4 График только инфицированных (I)

```
plt2 = @df df_ode plot(:t, :I,
    label=L"I(t)",
    xlabel="□□□□□, □□□",
    ylabel="□□□□□□□□□□ □□□□□□□□□□□□□□",
    title="□□□□□□□□ □□□□□ □□□□□□□□□□",
    color=:red,
    linewidth=2,
    fill=(0, 0.3, :red),
    grid=true,
    size=(800, 400))
```



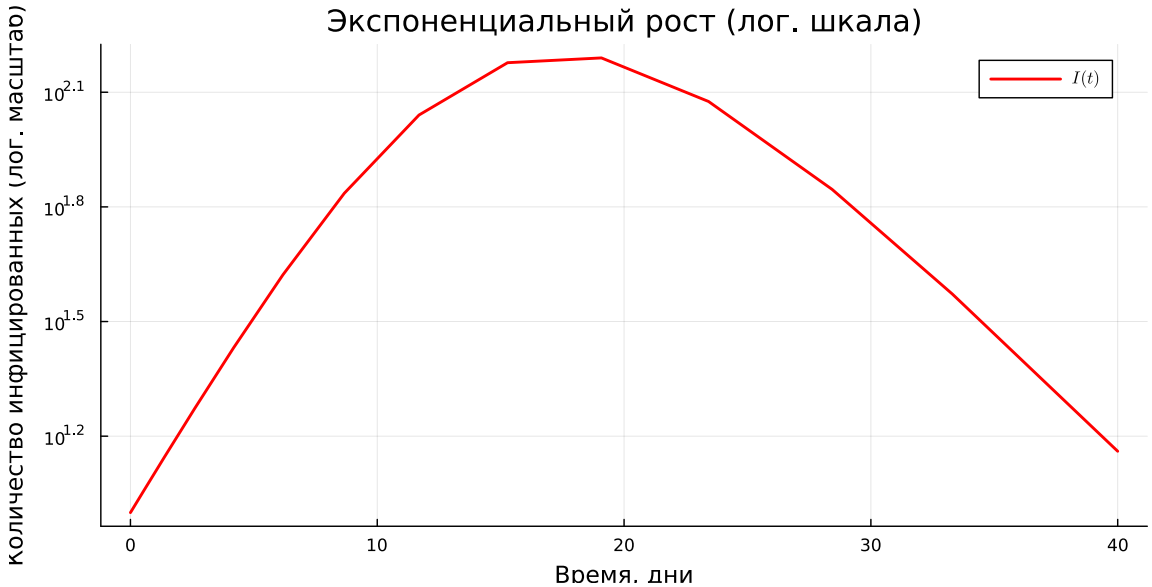
6.5 Отметка пика эпидемии

```
peak_idx = argmax(df_ode.I)
peak_time = df_ode.t[peak_idx]
peak_value = df_ode.I[peak_idx]
vline!(plt2, [peak_time], color=:black, linestyle=:dash, label=false
annotate!(plt2, peak_time, peak_value * 1.05,
    text("□□□: $(round(peak_value, digits=1)) □□ $(round(peak_time, d
    8, :top))
```



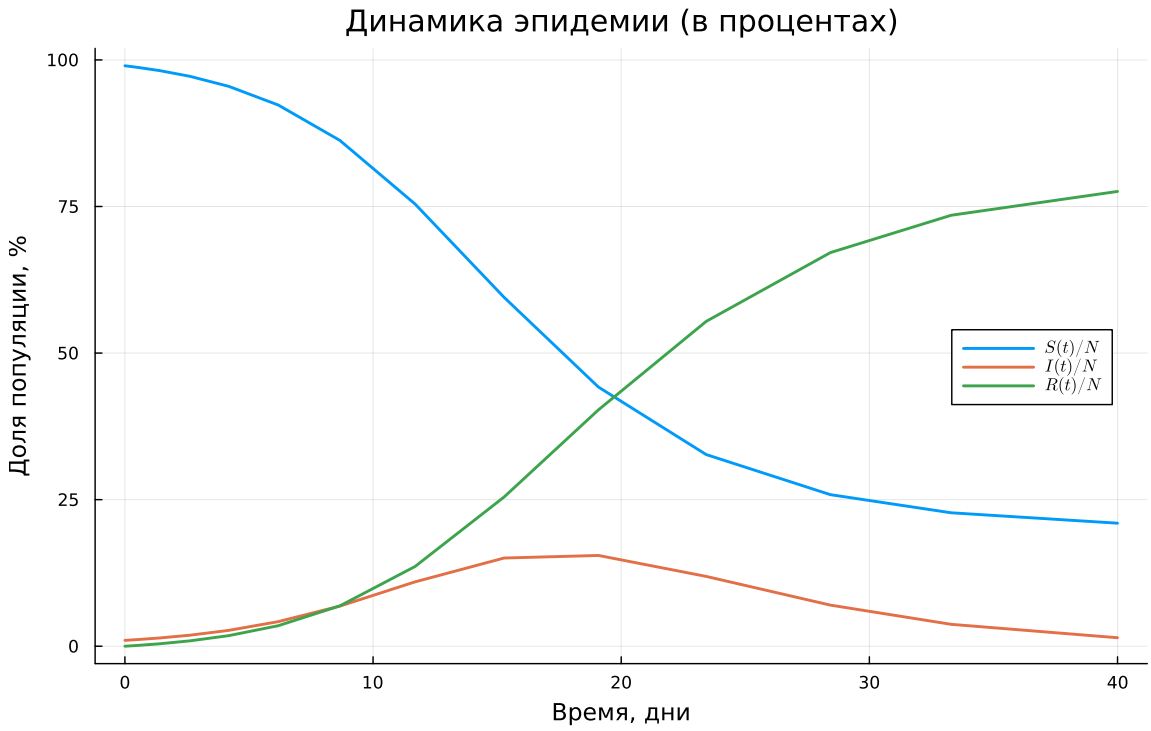
6.6 График в логарифмическом масштабе (для анализа экспоненциального роста)

```
plt3 = @df df_ode plot(:t, :I,
    label=L"I(t)",
    xlabel="□□□□□, □□□",
    ylabel="□□□□□□□□□□ □□□□□□□□□□□□□□ (□□□. □□□□□□□)",
    title="□□□□□□□□□□□□□□□□ □□□□ (□□□. □□□□□)",
    yscale=:log10,
    color=:red,
    linewidth=2,
    grid=true,
    size=(800, 400))
```



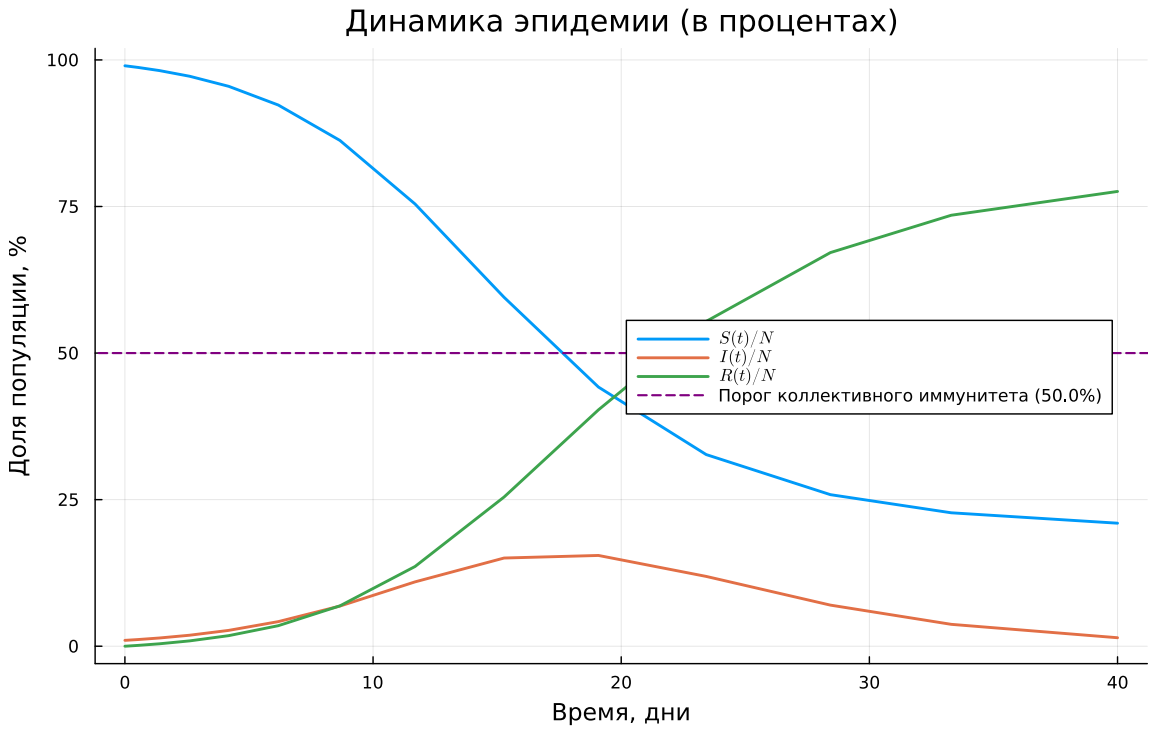
6.7 График долей населения (в процентах)

```
plt4 = @df df_ode plot(:t,
    [:S :I :R] ./ df_ode.N .* 100,
    label=[L"S(t)/N" L"I(t)/N" L"R(t)/N"],
    xlabel="□□□□□, □□□",
    ylabel="□□□□ □□□□□□□□□, %",
    title="□□□□□□□□ □□□□□□□□ (□ □□□□□□□□□)",
    linewidth=2,
    legend=:right,
    grid=true,
    size=(800, 500))
```



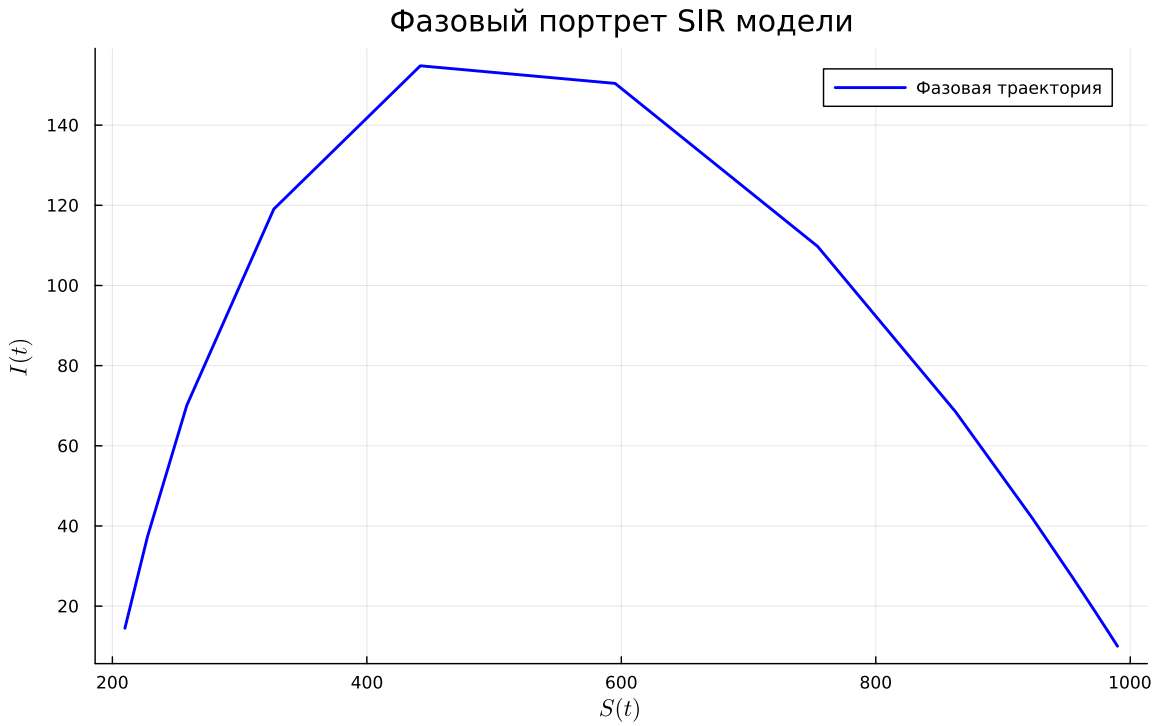
6.8 Горизонтальная линия для порога коллективного иммунитета

```
if R0 > 1
    herd_immunity_threshold = (1 - 1/R0) * 100
    hline!(plt4, [herd_immunity_threshold], color=:purple, linestyl
        label="□□□□□ □□□□□□□□□□□□ □□□□□□□□□□ ($(round(herd_immunity_thres
        linewidth=1.5)
end
```



6.9 Фазовый портрет (I vs S)

```
plt5 = plot(df_ode.S, df_ode.I,
    label="□□□□□□□ □□□□□□□□□□",
    xlabel=L"S(t)",
    ylabel=L"I(t)",
    title="□□□□□□□ □□□□□□□ SIR □□□□□□",
    color=:blue,
    linewidth=2,
    grid=true,
    size=(800, 500),
    legend=:topright)
```



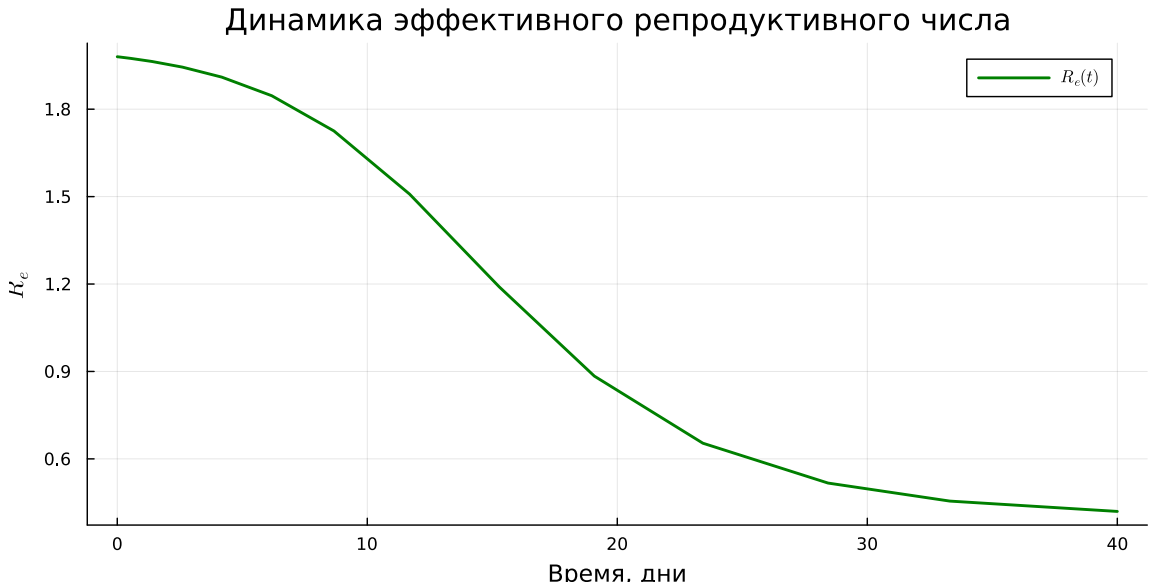
6.10 Добавление стрелок направления

```
for i in 1:50:length(df_ode.S)-1
    plot!(plt5, [df_ode.S[i], df_ode.S[i+1]], [df_ode.I[i], df_ode.
        arrow=:closed, color=:blue, alpha=0.5, label=false)
end
```

6.11 График R_e - эффективного репродуктивного числа

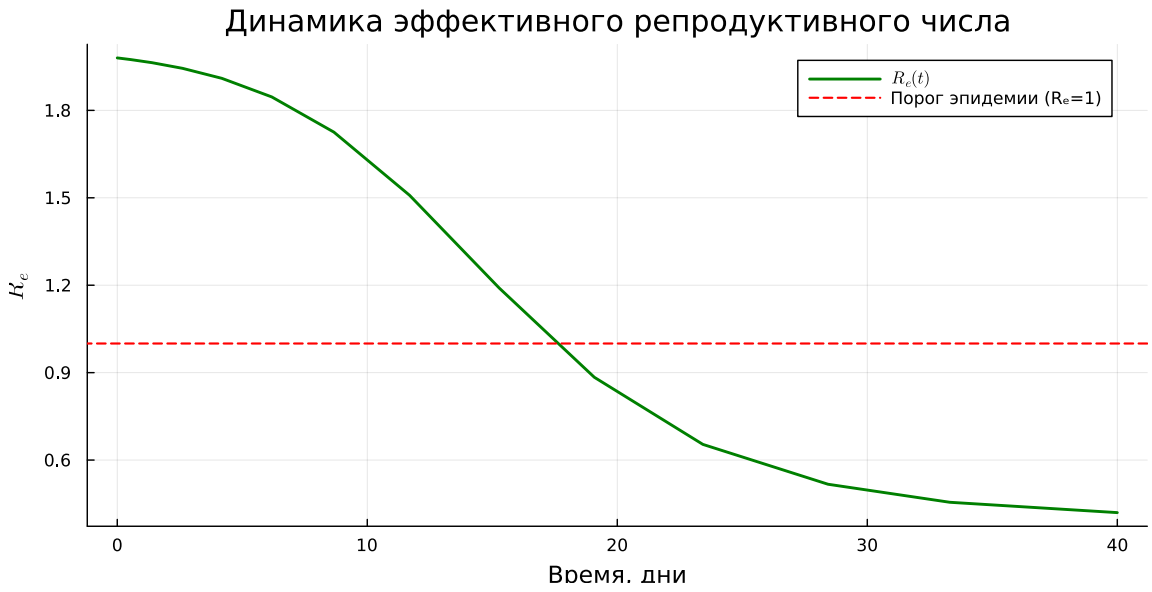
```
df_ode[!, :Re] = R0 .* df_ode.S ./ df_ode.N
plt6 = @df df_ode plot(:t, :Re,
    label=L"R_e(t)",
```

```
    xlabel="□□□□□, □□□",
    ylabel=L"R_e",
    title="□□□□□□□□ □□□□□□□□□□□□ □□□□□□□□□□□□□□□□ □□□□□",
    color=:green,
    linewidth=2,
    grid=true,
    size=(800, 400))
```



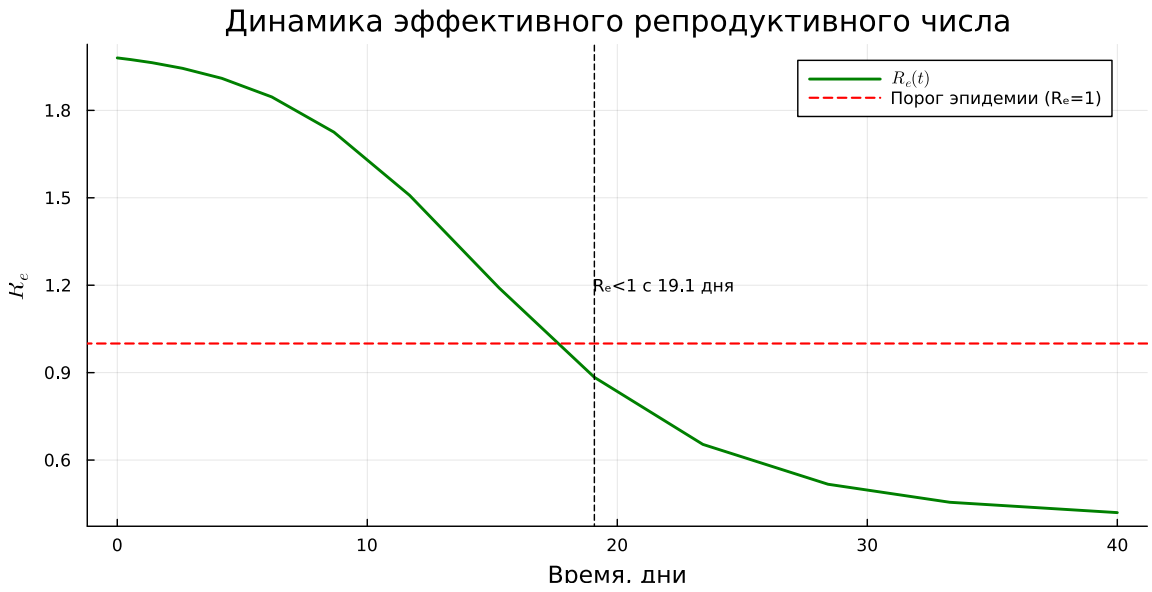
6.12 Горизонтальная линия на уровне 1

```
hline!(plt6, [1.0], color=:red, linestyle=:dash, label="□□□□□ □□□□□□□□
```



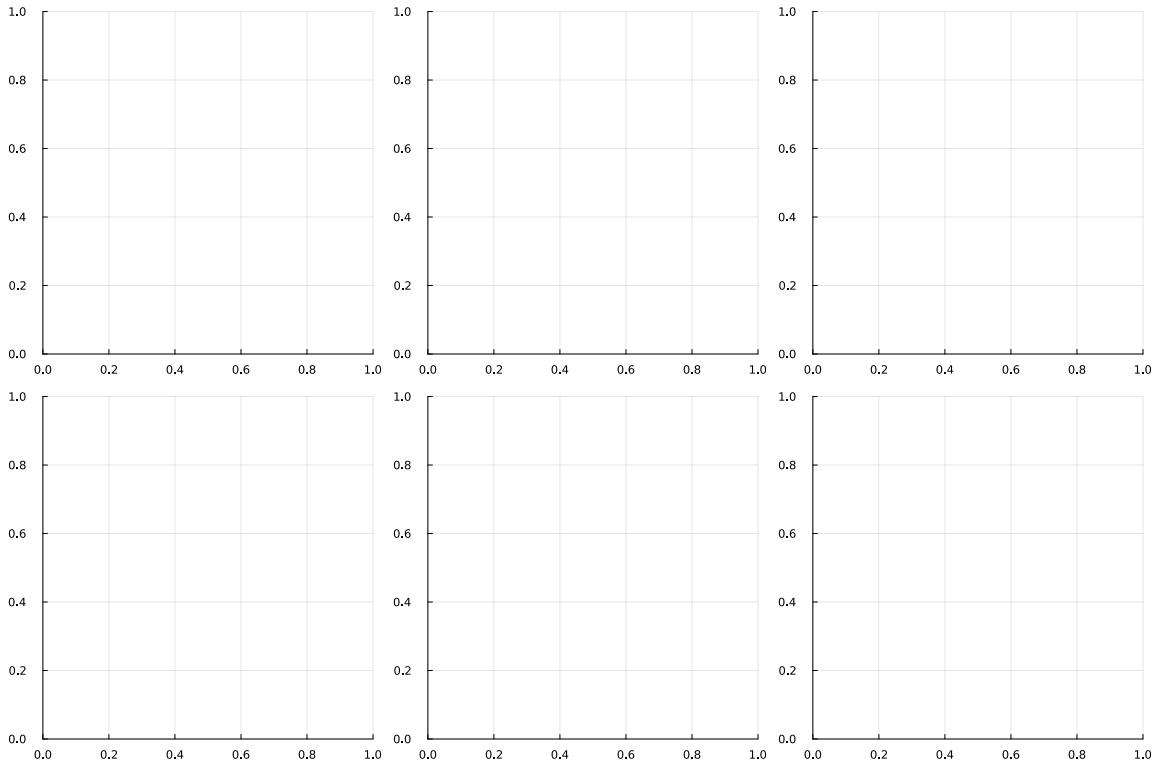
6.13 Отметка момента, когда R_e становится < 1

```
cross_idx = findfirst(x -> x < 1, df_ode.Re)
if !isnothing(cross_idx) && cross_idx > 1
    cross_time = df_ode.t[cross_idx]
    vline!(plt6, [cross_time], color=:black, linestyle=:dash, label
    annotate!(plt6, cross_time, 1.2,
        text("Rₑ<1 ⊠ $(round(cross_time, digits=1)) ⊠⊠⊠", 8, :left))
end
```



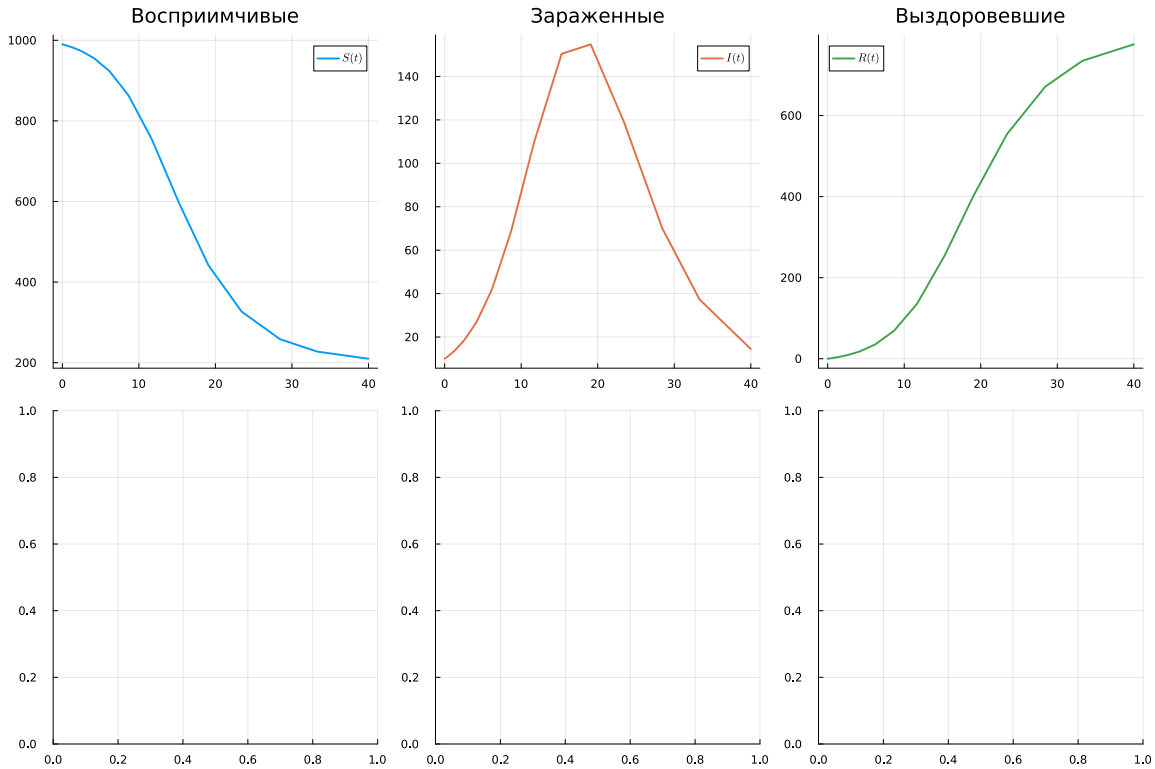
6.14 Компактный график всех кривых в одной панели

```
plt7 = plot(layout=(2, 3), size=(1200, 800))
```



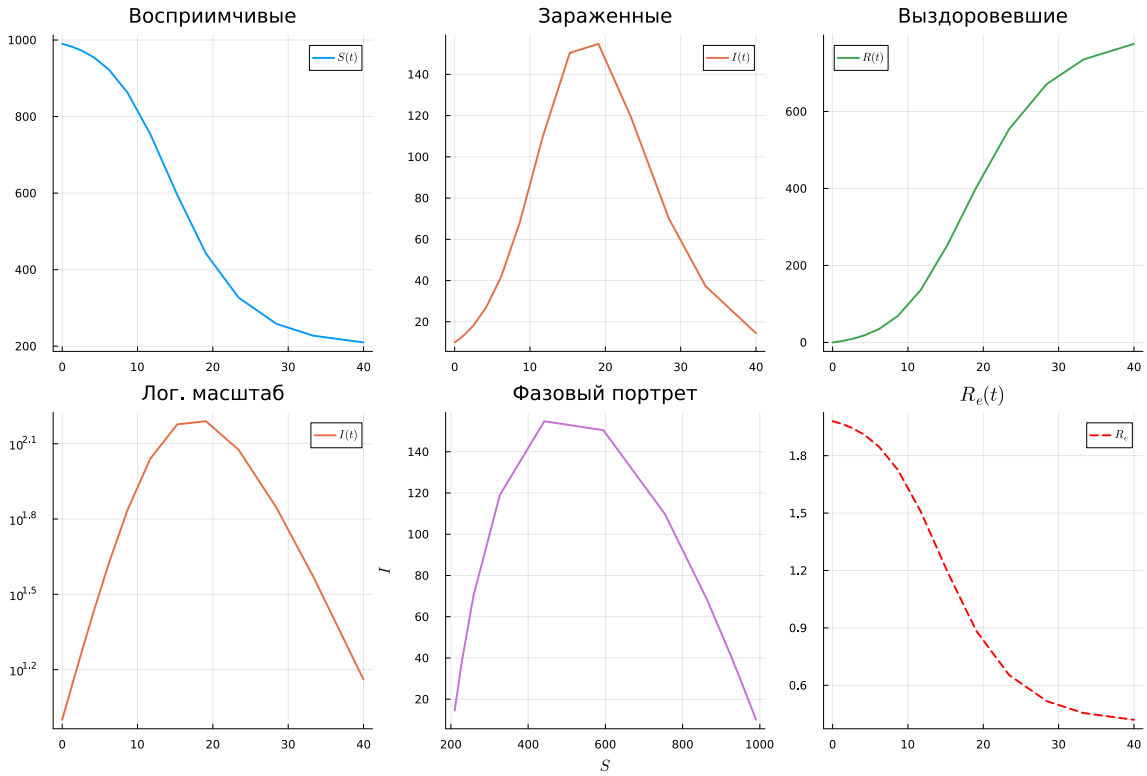
Верхний ряд

```
plot!(plt7[1], df_ode.t, df_ode.S, label=L"S(t)", color=1, linewidt
plot!(plt7[2], df_ode.t, df_ode.I, label=L"I(t)", color=2, linewidt
plot!(plt7[3], df_ode.t, df_ode.R, label=L"R(t)", color=3, linewidt
```



6.14.1 Нижний ряд

```
plot!(plt7[4], df_ode.t, df_ode.I, label=L"I(t)", color=2, linewidt
    yscale=:log10, title="□□□. □□□□□□□")
plot!(plt7[5], df_ode.S, df_ode.I, label=false, color=4, linewidth=
    title="□□□□□□□ □□□□□□□", xlabel=L"S", ylabel=L"I")
plot!(plt7[6], df_ode.t, df_ode.Re, label=L"R_e", color=:green, lin
    title=L"R_e(t)", hline=[1.0], linestyle=:dash, linecolor=:red)
```



6.15 Сохранение графиков

```
savefig(plt1, plotsdir(script_name, "sir_main.png"))
savefig(plt2, plotsdir(script_name, "sir_infected.png"))
savefig(plt3, plotsdir(script_name, "sir_log_scale.png"))
savefig(plt4, plotsdir(script_name, "sir_percentages.png"))
savefig(plt5, plotsdir(script_name, "sir_phase_portrait.png"))
savefig(plt6, plotsdir(script_name, "sir_effective_R.png"))
savefig(plt7, plotsdir(script_name, "sir_panel.png"))
```

```
"C:\\Users\\12232\\Documents\\GitHub\\2026-1--study--
simulation-modeling\\labs\\lab02\\project\\plots\\sir_panel.png"
```

6.16 Бенчмарк для оценки производительности

```
println("\n□□□□□□□□ □□□□□□□:")
@benchmark solve(prob_ode, dt = □t)
```

□□□□□□□□ □□□□□□□:

```
BenchmarkTools.Trial: 10000 samples with 6 evaluations per sample.
 Range (min … max):  6.050 □s …  10.441 ms  □ GC (min … max):  0.00% … 99.
 Time  (median):     10.217 □s              □ GC (median):    0.00%
 Time  (mean ± □):   12.255 □s ± 122.167 □s  □ GC (mean ± □):  13.66% ±  1.4

   ▄▄▄
```

```
if R0 > 1
    println("\n□□□□□□□□□□□□□ □□□□□□:")
    println("  - □□□□□ □□□□□□□□□□□□□ □□□□□□□□□□: ", round((1-1/R0)*100, di
    println("  - □□□□□□□□□□□□□ □□□ □□□ S/N = 1/R₀ = ", round(1/R0, digit
end
```

```
=== □□□□□□ □□□□□□□□□□□ ===
□□□□□ □□□□□□□□□□□ □□□□□□□□□ (□□□□□□□□): N = 1000.0
□□□□□□□ □□□□□ □□□□□□□□□□: I_max = 154.8
□□□□□ □□□□□□□□□□ □□□□: t_peak = 19.1 □□□□
□□□□□□□□ □□□□□ □□□□□□□□□□□□: R(∞) = 775.7
□□□□ □□□□□□□□□□□□: 77.6%

□□□□□□□□□□□□□ □□□□□□:
  - □□□□□ □□□□□□□□□□□□□ □□□□□□□□□□: 50.0%
  - □□□□□□□□□□□□□ □□□ □□□ S/N = 1/R₀ = 0.5
```

```
using DrWatson
@quickactivate "project"

using DifferentialEquations
using DataFrames
using StatsPlots
using LaTeXStrings
using Plots
using Statistics
using FFTW
```

```julia
script_name = splitext(basename(PROGRAM_FILE))[1]
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))
```

```
┌ Warning: DrWatson could not find find a project file by recursively cl
⊠ (given dir: C:\Users\12232)
└ @ DrWatson C:\Users\12232\.julia\packages\DrWatson\2QF5p\src\proje
```

```
"C:\\Users\\12232\\Documents\\GitHub\\2026-1--study--
simulation-modeling\\labs\\lab02\\project\\data\\"
```

7 Описание модели Лотки-Вольтерры

```
"""
□□□□□□ □□□□□-□□□□□□□□□ (□□□□□□-□□□□□□)

□□□□□□□ □□□□□□□□□:
  dx/dt = □x - □xy  # □□□□□□□□□ □□□□□□□□□ □□□□□
  dy/dt = □xy - □y  # □□□□□□□□□ □□□□□□□□□ □□□□□□□□

□□□:
  x - □□□□□□□□□ □□□□□ (□□□□□□□□, □□□□□)
  y - □□□□□□□□□ □□□□□□□□ (□□□□□□□□, □□□□)
  □ - □□□□□□□□□□□□ □□□□□□□ □□□□□ (□ □□□□□□□□□□ □□□□□□□□)
  □ - □□□□□□□□□□□ □□□□□□□□ □□□□□ □□□□□□□□□
  □ - □□□□□□□□□□□ □□□□□□□□ □□□□□□□□ □□ □□□□ □□□□□□□□ □□□□□
  □ - □□□□□□□□□□□□ □□□□□□□□□□ □□□□□□□□ (□ □□□□□□□□□□ □□□□□)
"""
```

"□□□□□□ □□□□□-□□□□□□□□□ (□□□□□□-□□□□□□)\n\n□□□□□□□ □□□□□□□□□:\n dx/dt = □x - □xy

Модель Лотки-Вольтерры — это фундаментальная математическая модель в экологии, описывающая динамику взаимодействия двух видов: хищников и жертв

Модель демонстрирует, как даже простая система взаимодействий может порождать сложные колебательные режимы, объясняя циклические изменения численности в природных экосистемах.

```
function lotka_volterra!(du, u, p, t)
    x, y = u          # x - □□□□□□, y - □□□□□□□
    □, □, □, □ = p    # □□□□□□□□□ □□□□□□

    @inbounds begin
    du[1] = □*x - □*x*y       # □□□□□□□□□ □□□ □□□□□
    du[2] = □*x*y - □*y       # □□□□□□□□□ □□□ □□□□□□□□
    end
    nothing
end
```

```
lotka_volterra! (generic function with 1 method)
```

8 Параметры модели и начальные условия

Классические параметры из литературы

```
p_lv = [0.1,    # □: □□□□□□□□ □□□□□□□□□□□ □□□□□
    0.02,   # □: □□□□□□□□ □□□□□□□□ □□□□□ □□□□□□□□□
    0.01,   # □: □□□□□□□□□□□ □□□□□□□□□ □□□□ (□□□□□) □ □□□□□□□□
    0.3]    # □: □□□□□□□□□□ □□□□□□□□
```

```
4-element Vector{Float64}:
 0.1
 0.02
 0.01
 0.3
```

Начальные условия: [жертвы, хищники]

```
u0_lv = [40.0, 9.0]  # □□□□□□□□□ □□□□□□□□□
```

```
2-element Vector{Float64}:
 40.0
  9.0
```

Временные параметры

```julia
tspan_lv = (0.0, 200.0)  # □□□□□□□□□□□□ □□□□□□□□□
dt_lv = 0.01               # □□□ □□□□□□□□□□□□□□
```

```
0.01
```

Создание и решение задачи

```julia
prob_lv = ODEProblem(lotka_volterra!, u0_lv, tspan_lv, p_lv)
sol_lv = solve(prob_lv,
               dt = dt_lv,
               Tsit5(),            # □□□□□ 5-□□ □□□□□□□
               reltol=1e-8,        # □□□□□□□□□□□□□ □□□□□□□□
               abstol=1e-10,       # □□□□□□□□□□ □□□□□□□□
               saveat=0.1,         # □□□□□□□□□ □□□□□□ 0.1 □□□□□□□ □□□□□□□
               dense=true          # □□□□□□□□ □□□□□□□ □□□□□ □□□ □□□□□□□□□□□□
)
```

```
┌ Warning: Verbosity toggle: dense_output_saveat
□ Dense output is incompatible with saveat. Please use the SavingCallb
└ @ OrdinaryDiffEqCore C:\Users\12232\.julia\packages\OrdinaryDiffEq
```

```
retcode: Success
Interpolation: specialized 4th order "free" interpolation
t: 2001-element Vector{Float64}:
   0.0
   0.1
   0.2
   0.3
   0.4
   0.5
```

```
   0.6
   0.7
   0.8
   0.9
   1.0
   1.1
   1.2
   ⋮
 198.9
 199.0
 199.1
 199.2
 199.3
 199.4
 199.5
 199.6
 199.7
 199.8
 199.9
 200.0
u: 2001-element Vector{Vector{Float64}}:
 [40.0, 9.0]
 [39.67773201890384, 9.088990237207637]
 [39.35113789503168, 9.175882726306998]
 [39.02053587808482, 9.260562174341327]
 [38.68624820281499, 9.342916311942014]
 [38.34860018759226, 9.422836265415745]
 [38.007919320045765, 9.5002169187062]
 [37.664534369328166, 9.574957250330058]
```

[37.31877446654823, 9.646960666854817]

[36.9709682176784, 9.716135308317774]

[36.62144287329974, 9.782394321389948]

[36.270523365122514, 9.845656154267312]

[35.91853163311434, 9.905844751305578]

[17.88967563421412, 5.712401894068302]

[17.86543624231875, 5.643570483722269]

[17.843673453877365, 5.5754402198919735]

[17.824353144929315, 5.508019318506856]

[17.807442037033756, 5.441315154551648]

[17.792907686320664, 5.375334300040311]

[17.780718473179327, 5.310082577777411]

[17.770843588540227, 5.24556508481173]

[17.76325301314863, 5.181786245480946]

[17.75791751745926, 5.118749811721092]

[17.754808657966564, 5.056458873592464]

[17.75389874795971, 4.994915937053478]

9 Подготовка данных

```
df_lv = DataFrame()
df_lv[!, :t] = sol_lv.t
df_lv[!, :prey] = [u[1] for u in sol_lv.u]  # □□□□□□
df_lv[!, :predator] = [u[2] for u in sol_lv.u]  # □□□□□□□
```

```
2001-element Vector{Float64}:
 9.0
 9.088990237207637
 9.175882726306998
 9.260562174341327
 9.342916311942014
 9.422836265415745
 9.5002169187062
 9.574957250330058
 9.646960666854817
 9.716135308317774
 9.782394321389948
 9.845656154267312
 9.905844751305578
 □
 5.712401894068302
```

5.643570483722269

5.5754402198919735

5.508019318506856

5.441315154551648

5.375334300040311

5.310082577777411

5.24556508481173

5.181786245480946

5.118749811721092

5.056458873592464

4.994915937053478

10 Рассчет производных для анализа

```
df_lv[!, :dprey_dt] = p_lv[1] .* df_lv.prey .- p_lv[2] .* df_lv.prey
df_lv[!, :dpredator_dt] = p_lv[3] .* df_lv.prey .* df_lv.predator .
```

```
2001-element Vector{Float64}:
  0.9000000000000004
  0.8796081183812872
  0.8580494468233599
  0.8353523334488111
  0.8115489002365717
  0.7866749261310111
  0.7607697060793308
  0.7338758893000268
  0.706039294083721
  0.677308704329461
  0.6477356516317538
  0.6173741696029547
  0.5862805551332175
  ⋮
 -0.6917903984489675
 -0.6848226585569588
 -0.6777687195139261
```

-0.6706369809304658

-0.6634355041663761

-0.6561720201747897

-0.648853939485278

-0.6414883588865488

-0.6340820722589682

-0.6266415740257949

-0.6191730642026259

-0.611682463105866

11 Вывод информации о модели

```
println("="^60)
println("□□□□□□ □□□□□-□□□□□□□□□ (□□□□□□-□□□□□□)")
println("="^60)
println("\n□□□□□□□□□ □□□□□□:")
println("□ (□□□□□□□□ □□□□□□□□□□□ □□□□□) = ", p_lv[1])
println("□ (□□□□□□□□ □□□□□□□□ □□□□□) = ", p_lv[2])
println("□ (□□□□□□□□□□□ □□□□□□□□□) = ", p_lv[3])
println("□ (□□□□□□□□□□ □□□□□□□□) = ", p_lv[4])
println("\n□□□□□□□□□ □□□□□□□:")
println("□□□□□□ (x₀) = ", u0_lv[1])
println("□□□□□□□ (y₀) = ", u0_lv[2])
```

```
============================================================
□□□□□□ □□□□□-□□□□□□□□□ (□□□□□□-□□□□□□)
============================================================

□□□□□□□□□ □□□□□□:
□ (□□□□□□□□ □□□□□□□□□□□ □□□□□) = 0.1
□ (□□□□□□□□ □□□□□□□□ □□□□□) = 0.02
□ (□□□□□□□□□□□ □□□□□□□□□) = 0.01
□ (□□□□□□□□□□ □□□□□□□□) = 0.3
```

[illegible] [illegible]:

[illegible] (x_0) = 40.0

[illegible] (y_0) = 9.0

12 Стационарные точки (нулевые изоклины)

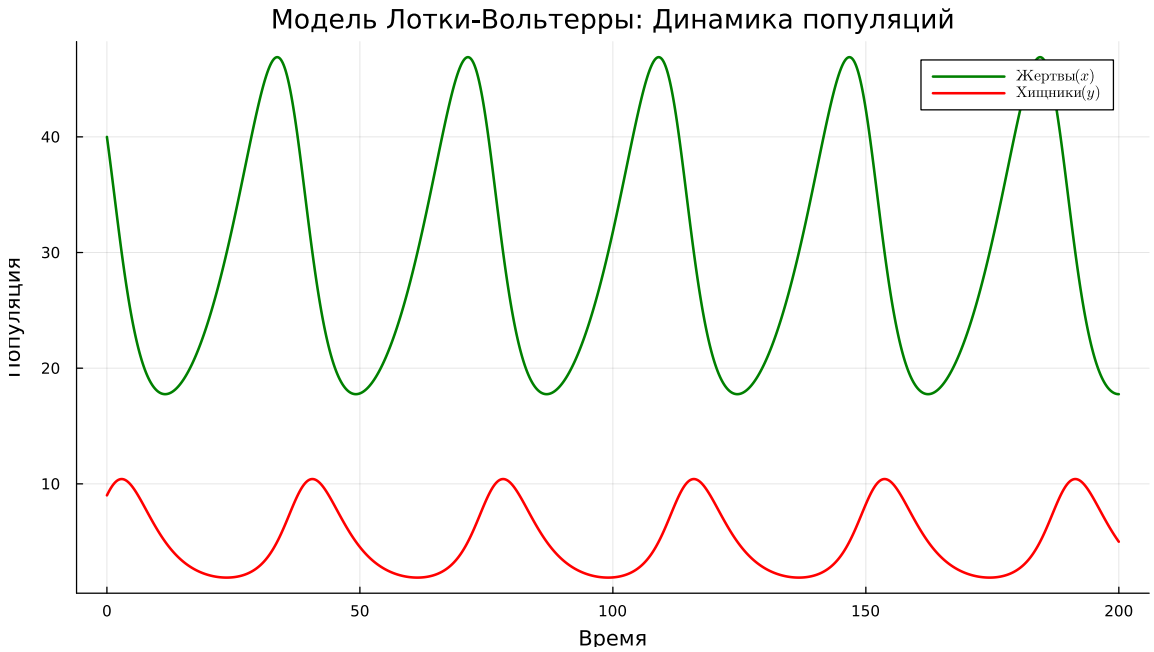
```
x_star = p_lv[4] / p_lv[3]  # □□□□□□□□□□□□ □□□□□ □□□ □□□□□
y_star = p_lv[1] / p_lv[2]  # □□□□□□□□□□□□ □□□□□ □□□ □□□□□□□□
println("\n□□□□□□□□□□□□ □□□□□ (□□□□□□□□□ □□□□□□□□□□):")
println("x* = □/□ = ", round(x_star, digits=3))
println("y* = □/□ = ", round(y_star, digits=3))
```

```
□□□□□□□□□□□□ □□□□□ (□□□□□□□□□ □□□□□□□□□□):
x* = □/□ = 30.0
y* = □/□ = 5.0
```

13 Построение графиков

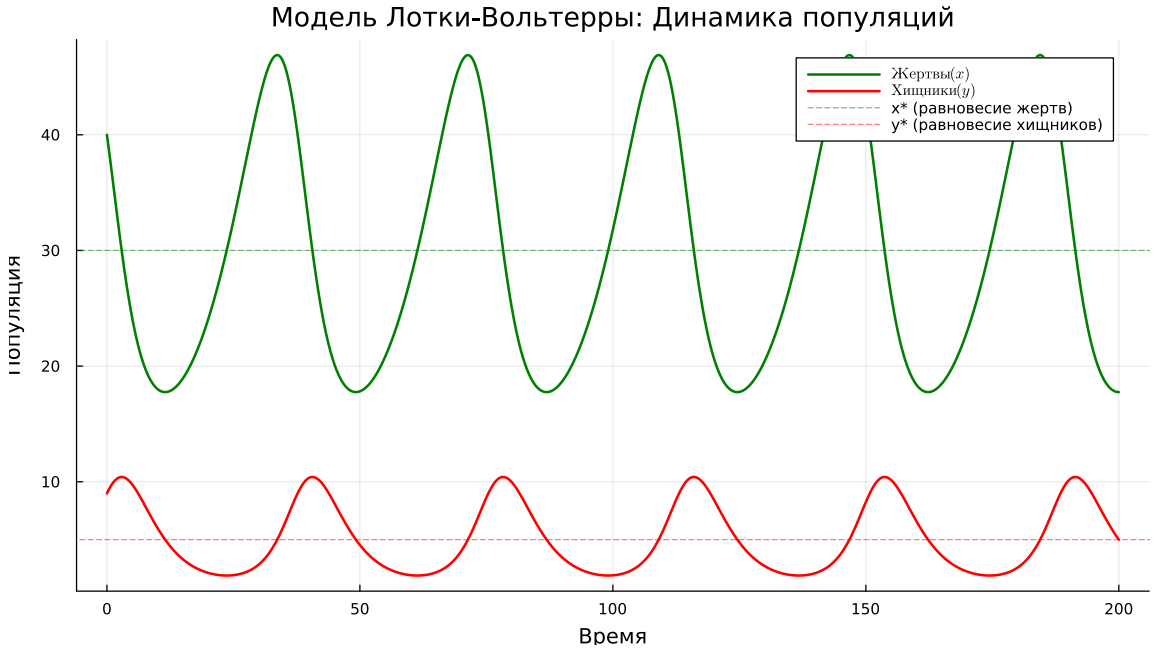
13.1 График 1: Динамика популяций во времени

```
plt1 = plot(df_lv.t, [df_lv.prey df_lv.predator],
    label=[L"□□□□□□ (x)" L"□□□□□□□ (y)"],
    xlabel="□□□□□",
    ylabel="□□□□□□□□□",
    title="□□□□□□ □□□□□-□□□□□□□□□: □□□□□□□□ □□□□□□□□□",
    linewidth=2,
    legend=:topright,
    grid=true,
    size=(900, 500),
    color=[:green :red])
```



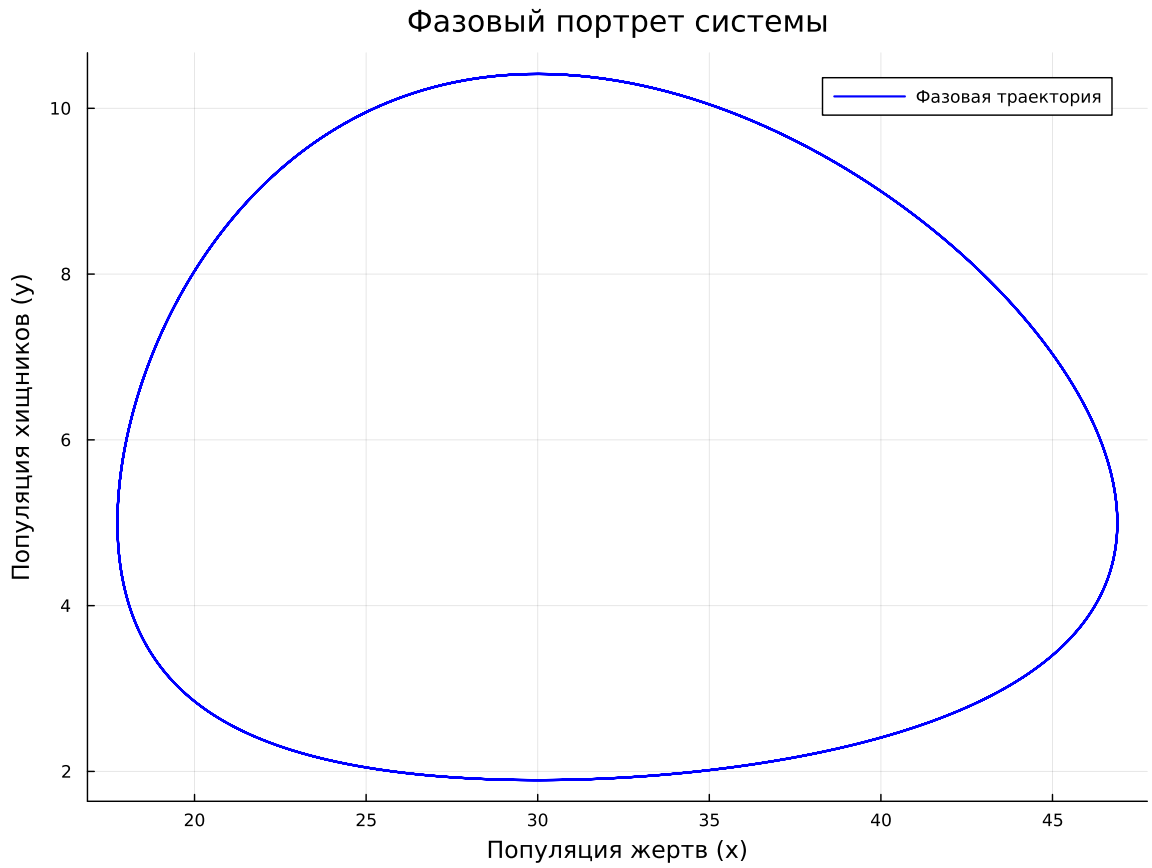
Добавление стационарных уровней

```
hline!(plt1, [x_star], color=:green, linestyle=:dash, alpha=0.5, la
hline!(plt1, [y_star], color=:red, linestyle=:dash, alpha=0.5, labe
```



13.2 График 2: Фазовый портрет (хищники vs жертвы)

```
plt2 = plot(df_lv.prey, df_lv.predator,
    label="□□□□□□□ □□□□□□□□□□",
    xlabel="□□□□□□□□□ □□□□□ (x)",
    ylabel="□□□□□□□□□ □□□□□□□□ (y)",
    title="□□□□□□□ □□□□□□□ □□□□□□□",
    color=:blue,
    linewidth=1.5,
    grid=true,
    size=(800, 600),
    legend=:topright)
```

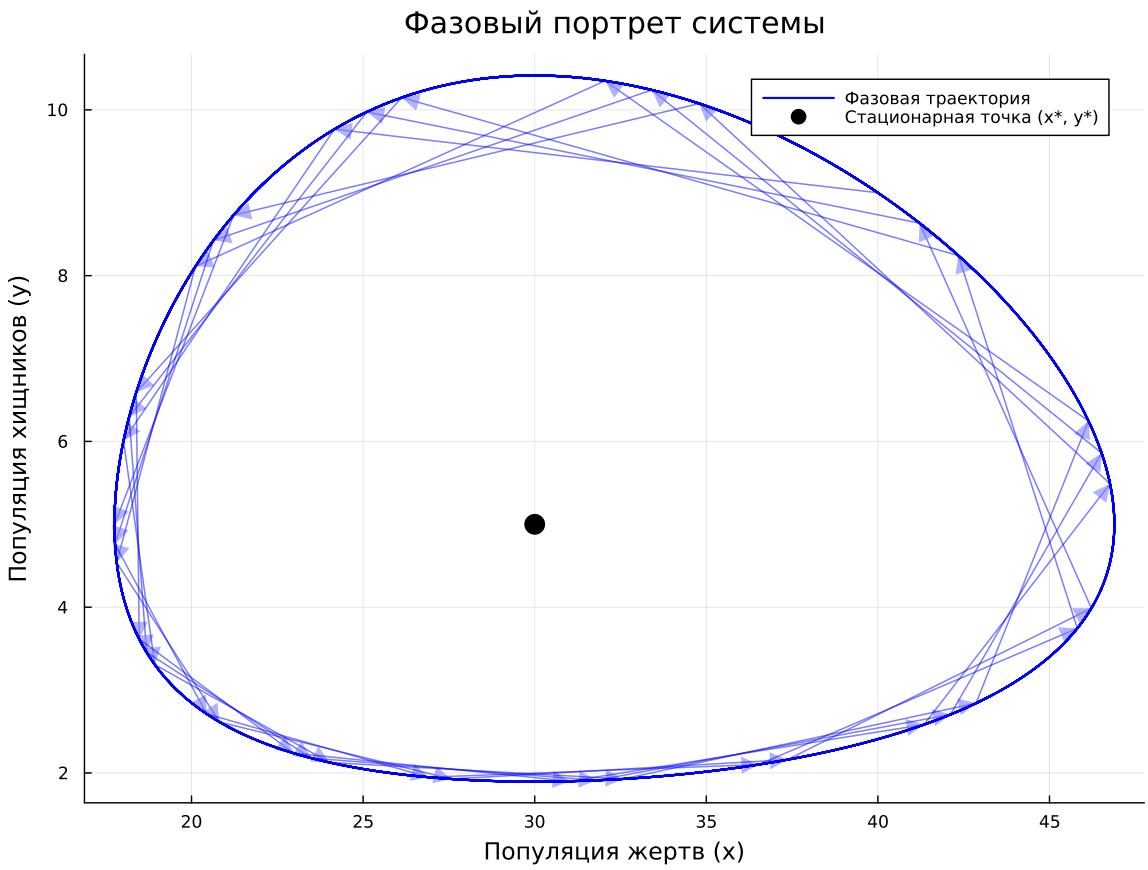


Добавление стрелок направления на фазовом портрете

```
step = 50  # [illegible] [illegible] [illegible] [illegible]
for i in 1:step:length(df_lv.prey)-step
    plot!(plt2, [df_lv.prey[i], df_lv.prey[i+step]],
      [df_lv.predator[i], df_lv.predator[i+step]],
      arrow=:closed, color=:blue, alpha=0.3, label=false)
end
```

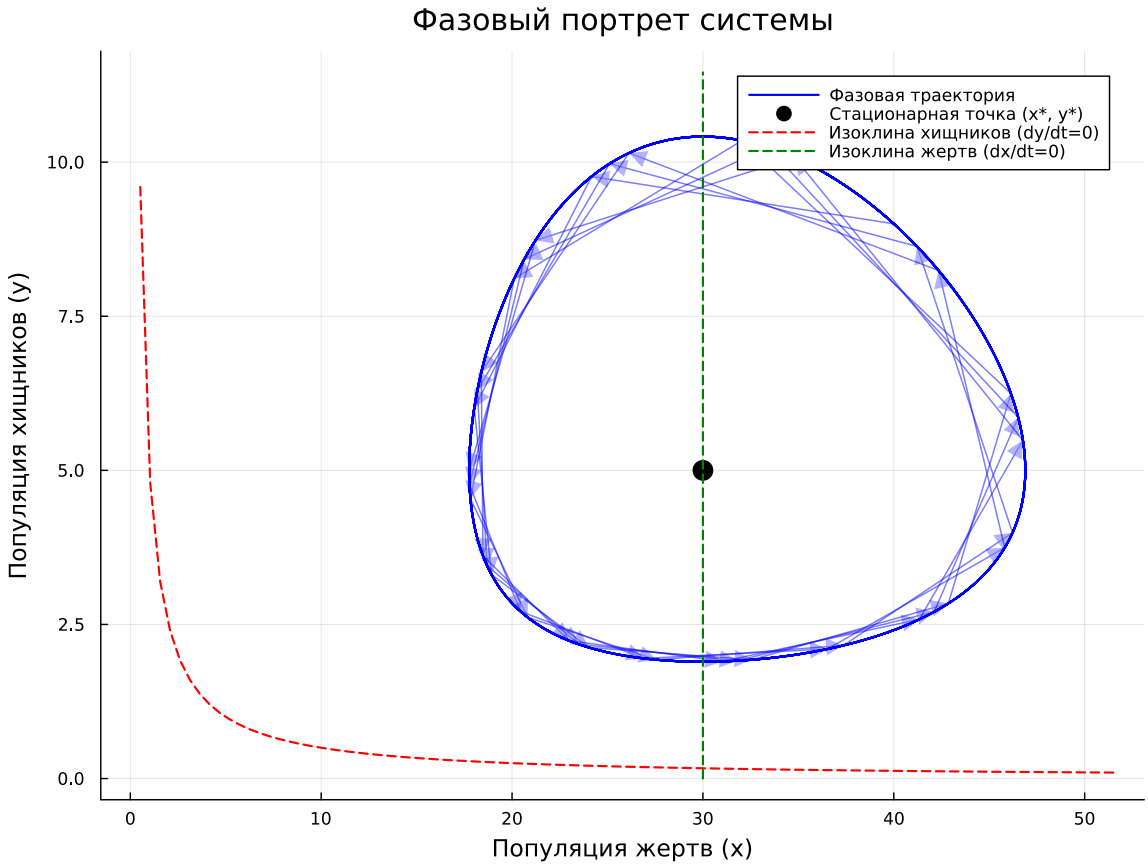
Добавление стационарной точки

```
scatter!(plt2, [x_star], [y_star],
    color=:black, markersize=8, label="[illegible] [illegible] (x*, y*)")
```



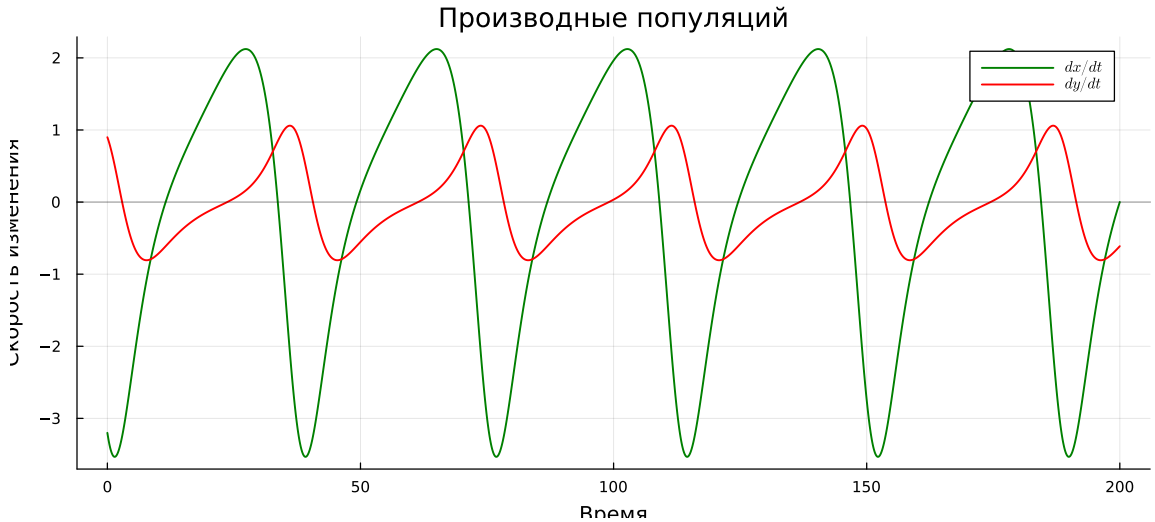
Изоклины (нулевого роста)

```
x_range = LinRange(0, maximum(df_lv.prey)*1.1, 100)
y_nullcline = p_lv[1] ./ (p_lv[2] .* x_range)  # y-□□□□□□□ (dy/dt =
plot!(plt2, x_range, y_nullcline,
    color=:red, linestyle=:dash, linewidth=1.5, label="□□□□□□□ □□□□□□□

y_range = LinRange(0, maximum(df_lv.predator)*1.1, 100)
x_nullcline = p_lv[4] ./ (p_lv[3] .* ones(length(y_range)))  # x-□□□
plot!(plt2, x_nullcline, y_range,
    color=:green, linestyle=:dash, linewidth=1.5, label="□□□□□□□ □□□□
```



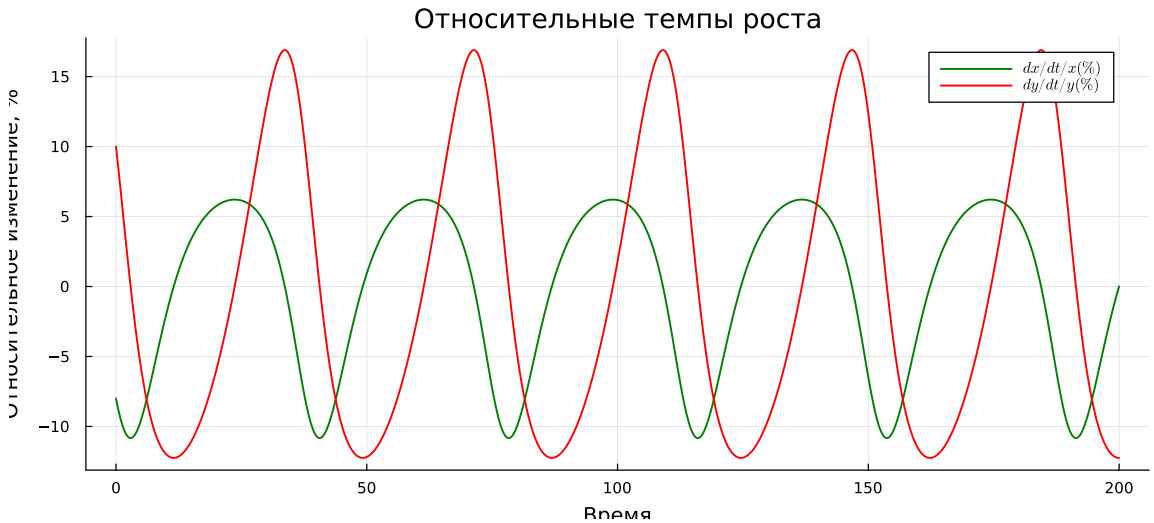
13.3 График 3: Производные (скорости изменения)

```
plt3 = plot(df_lv.t, [df_lv.dprey_dt df_lv.dpredator_dt],
    label=[L"dx/dt" L"dy/dt"],
    xlabel="□□□□□",
    ylabel="□□□□□□□□ □□□□□□□□□",
    title="□□□□□□□□□□□ □□□□□□□□□",
    linewidth=1.5,
    legend=:topright,
    grid=true,
    size=(900, 400),
    color=[:green :red])

hline!(plt3, [0], color=:black, linestyle=:solid, alpha=0.3, label=
```



13.4 График 4: Относительные изменения (в %)

```
df_lv[!, :prey_pct_change] = df_lv.dprey_dt ./ df_lv.prey .* 100
df_lv[!, :predator_pct_change] = df_lv.dpredator_dt ./ df_lv.predat

plt4 = plot(df_lv.t, [df_lv.prey_pct_change df_lv.predator_pct_chan
    label=[L"dx/dt / x (\%)" L"dy/dt / y (\%)"],
    xlabel="□□□□□",
    ylabel="□□□□□□□□□□□□□ □□□□□□□□□, %",
    title="□□□□□□□□□□□□□ □□□□□ □□□□□",
    linewidth=1.5,
    legend=:topright,
    grid=true,
    size=(900, 400),
    color=[:green :red])
```



13.5 График 5: Спектральный анализ (быстрое преобразование Фурье)

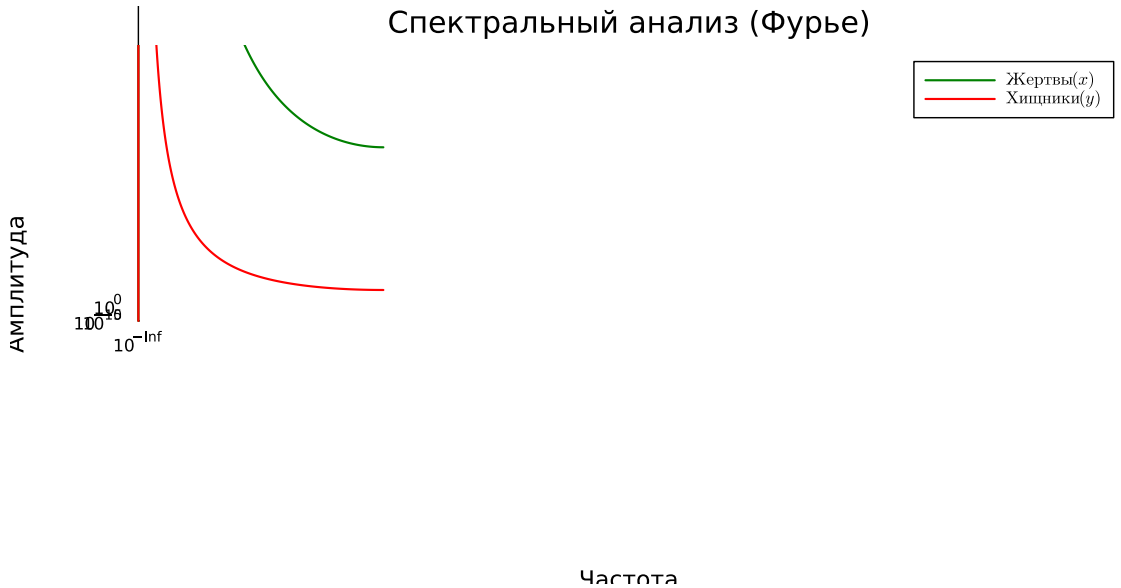
```
function compute_fft(signal, dt)
    n = length(signal)
    spectrum = abs.(rfft(signal)) # □□□□□□□□□□ rfft □□□ □□□□□□□□□□□□ □□□□□
    freq = rfftfreq(n, 1/dt) # □□□□□□□□□□□□□□□ □□□□□□□ □□□ rfft
    return freq, spectrum
end
```

```
compute_fft (generic function with 1 method)
```

Вычисление периодов колебаний

```
freq_prey, spectrum_prey = compute_fft(df_lv.prey .- mean(df_lv.prey
freq_predator, spectrum_predator = compute_fft(df_lv.predator .- mea

plt5 = plot(freq_prey, [spectrum_prey spectrum_predator],
    label=[L"□□□□□□ (x)" L"□□□□□□□ (y)"],
    xlabel="□□□□□□□",
    ylabel="□□□□□□□□□",
    title="□□□□□□□□□□□□ □□□□□□ (□□□□□)",
    linewidth=1.5,
    xscale=:log10,
    yscale=:log10,
    legend=:topright,
    grid=true,
    size=(800, 400),
    color=[:green :red])
```

```
┌ Warning: No strict ticks found
└ @ PlotUtils C:\Users\12232\.julia\packages\PlotUtils\HX80C\src\tic
┌ Warning: No strict ticks found
└ @ PlotUtils C:\Users\12232\.julia\packages\PlotUtils\HX80C\src\tic
┌ Warning: Invalid negative or zero value 0.0 found at series index 1 fo
└ @ Plots C:\Users\12232\.julia\packages\Plots\GIume\src\utils.jl:10
┌ Warning: Invalid negative or zero value 0.0 found at series index 1 fo
└ @ Plots C:\Users\12232\.julia\packages\Plots\GIume\src\utils.jl:10
┌ Warning: No strict ticks found
└ @ PlotUtils C:\Users\12232\.julia\packages\PlotUtils\HX80C\src\tic
┌ Warning: No strict ticks found
└ @ PlotUtils C:\Users\12232\.julia\packages\PlotUtils\HX80C\src\tic
┌ Warning: Invalid negative or zero value 0.0 found at series index 1 fo
└ @ Plots C:\Users\12232\.julia\packages\Plots\GIume\src\utils.jl:10
┌ Warning: Invalid negative or zero value 0.0 found at series index 1 fo
└ @ Plots C:\Users\12232\.julia\packages\Plots\GIume\src\utils.jl:10
```




```
┌ Warning: No strict ticks found
└ @ PlotUtils C:\Users\12232\.julia\packages\PlotUtils\HX80C\src\tic
```

```
┌ Warning: No strict ticks found
└ @ PlotUtils C:\Users\12232\.julia\packages\PlotUtils\HX80C\src\tic
┌ Warning: No strict ticks found
└ @ PlotUtils C:\Users\12232\.julia\packages\PlotUtils\HX80C\src\tic
┌ Warning: Invalid negative or zero value 0.0 found at series index 1 fo
└ @ Plots C:\Users\12232\.julia\packages\Plots\GIume\src\utils.jl:10
┌ Warning: Invalid negative or zero value 0.0 found at series index 1 fo
└ @ Plots C:\Users\12232\.julia\packages\Plots\GIume\src\utils.jl:10
┌ Warning: No strict ticks found
└ @ PlotUtils C:\Users\12232\.julia\packages\PlotUtils\HX80C\src\tic
┌ Warning: No strict ticks found
└ @ PlotUtils C:\Users\12232\.julia\packages\PlotUtils\HX80C\src\tic
┌ Warning: Invalid negative or zero value 0.0 found at series index 1 fo
└ @ Plots C:\Users\12232\.julia\packages\Plots\GIume\src\utils.jl:10
┌ Warning: Invalid negative or zero value 0.0 found at series index 1 fo
└ @ Plots C:\Users\12232\.julia\packages\Plots\GIume\src\utils.jl:10
```

Нахождение доминирующих частот

```julia
if length(spectrum_prey) > 0
    idx_prey = argmax(spectrum_prey[2:end]) + 1  # □□□□□□□□□□ □□□□□□□ □
    dominant_freq_prey = freq_prey[idx_prey]
    period_prey = 1/dominant_freq_prey
    println("\n□□□□□□□□□□□ □□□□□□□ □□□□□□□□□ □□□□□: ", round(dominant_freq
    println("□□□□□□ □□□□□□□□□ □□□□□: ", round(period_prey, digits=2), "
end
```

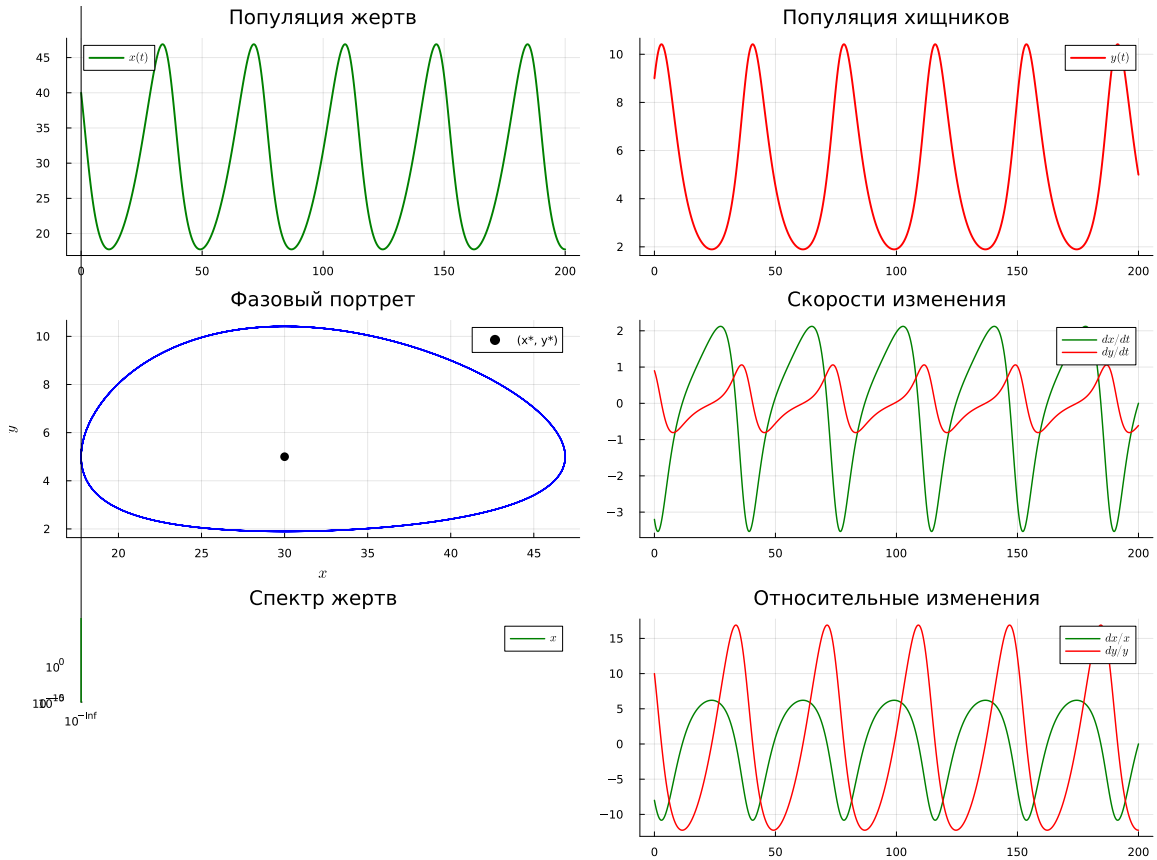
```
□□□□□□□□□□□ □□□□□□□ □□□□□□□□□ □□□□□: 0.2499 □□
□□□□□□ □□□□□□□□□ □□□□□: 4.0 □□□□□□ □□□□□□□
```

13.6 График 6: Компактная панель всех графиков

```
plt6 = plot(layout=(3, 2), size=(1200, 900))

plot!(plt6[1], df_lv.t, df_lv.prey, label=L"x(t)", color=:green, li
      title="□□□□□□□□ □□□□□", grid=true)
plot!(plt6[2], df_lv.t, df_lv.predator, label=L"y(t)", color=:red,
      title="□□□□□□□□ □□□□□□□□", grid=true)
plot!(plt6[3], df_lv.prey, df_lv.predator, label=false, color=:blue
      title="□□□□□□□ □□□□□□□", xlabel=L"x", ylabel=L"y", grid=true)
scatter!(plt6[3], [x_star], [y_star], color=:black, markersize=5, la
plot!(plt6[4], df_lv.t, [df_lv.dprey_dt df_lv.dpredator_dt],
      label=[L"dx/dt" L"dy/dt"], color=[:green :red], linewidth=1.5
      title="□□□□□□□□ □□□□□□□□□", grid=true, legend=:topright)
plot!(plt6[5], freq_prey, spectrum_prey, label=L"x", color=:green,
      title="□□□□□□ □□□□□", xscale=:log10, yscale=:log10, grid=true)
plot!(plt6[6], df_lv.t, [df_lv.prey_pct_change df_lv.predator_pct_cl
      label=[L"dx/x" L"dy/y"], color=[:green :red], linewidth=1.5,
      title="□□□□□□□□□□□□□ □□□□□□□□□", grid=true, legend=:topright)
```

```
┌ Warning: No strict ticks found
└ @ PlotUtils C:\Users\12232\.julia\packages\PlotUtils\HX80C\src\tic
┌ Warning: No strict ticks found
└ @ PlotUtils C:\Users\12232\.julia\packages\PlotUtils\HX80C\src\tic
┌ Warning: Invalid negative or zero value 0.0 found at series index 1 fo
└ @ Plots C:\Users\12232\.julia\packages\Plots\GIume\src\utils.jl:10
┌ Warning: No strict ticks found
└ @ PlotUtils C:\Users\12232\.julia\packages\PlotUtils\HX80C\src\tic
┌ Warning: No strict ticks found
```

```
└ @ PlotUtils C:\Users\12232\.julia\packages\PlotUtils\HX80C\src\tic
┌ Warning: Invalid negative or zero value 0.0 found at series index 1 fo
└ @ Plots C:\Users\12232\.julia\packages\Plots\GIume\src\utils.jl:10
┌ Warning: No strict ticks found
└ @ PlotUtils C:\Users\12232\.julia\packages\PlotUtils\HX80C\src\tic
┌ Warning: No strict ticks found
└ @ PlotUtils C:\Users\12232\.julia\packages\PlotUtils\HX80C\src\tic
┌ Warning: No strict ticks found
└ @ PlotUtils C:\Users\12232\.julia\packages\PlotUtils\HX80C\src\tic
┌ Warning: Invalid negative or zero value 0.0 found at series index 1 fo
└ @ Plots C:\Users\12232\.julia\packages\Plots\GIume\src\utils.jl:10
```




```
┌ Warning: No strict ticks found
└ @ PlotUtils C:\Users\12232\.julia\packages\PlotUtils\HX80C\src\tic
```

```
┌ Warning: No strict ticks found
└ @ PlotUtils C:\Users\12232\.julia\packages\PlotUtils\HX80C\src\tic
┌ Warning: Invalid negative or zero value 0.0 found at series index 1 fo
└ @ Plots C:\Users\12232\.julia\packages\Plots\GIume\src\utils.jl:10
```

14 Анализ результатов

```
println("\n" * "="^60)
println("[illegible] [illegible]")
println("="^60)

println("\n[illegible] [illegible]:")
println("[illegible]: min = ", round(minimum(df_lv.prey), digits=2),
        ", max = ", round(maximum(df_lv.prey), digits=2),
        ", mean = ", round(mean(df_lv.prey), digits=2))
println("[illegible]: min = ", round(minimum(df_lv.predator), digits=2),
        ", max = ", round(maximum(df_lv.predator), digits=2),
        ", mean = ", round(mean(df_lv.predator), digits=2))
```

```
============================================================
[illegible] [illegible]
============================================================

[illegible] [illegible]:
[illegible]: min = 17.75, max = 46.89, mean = 29.71
[illegible]: min = 1.9, max = 10.41, mean = 5.2
```

Упрощенный анализ колебаний без поиска максимумов Вместо сложного анализа

сдвига фаз, просто посчитаем основные характеристики

Находим время первого пика жертв (простой алгоритм)

```
function find_first_peak(signal, time)
    for i in 2:length(signal)-1
        if signal[i] > signal[i-1] && signal[i] > signal[i+1]
            return time[i], signal[i]
        end
    end
    return NaN, NaN
end

peak_time_prey, peak_value_prey = find_first_peak(df_lv.prey, df_lv
peak_time_predator, peak_value_predator = find_first_peak(df_lv.pre

if !isnan(peak_time_prey) && !isnan(peak_time_predator)
    phase_shift = peak_time_predator - peak_time_prey
    println("\n□□□□□□ □□□□□□□□□:")
    println("□□□□□□ □□□ □□□□□: □□□□□ = ", round(peak_time_prey, digits=2
            ", □□□□□□□□ = ", round(peak_value_prey, digits=2))
    println("□□□□□□ □□□ □□□□□□□□: □□□□□ = ", round(peak_time_predator, d
            ", □□□□□□□□ = ", round(peak_value_predator, digits=2))
    println("□□□□□ □□□ (□□□□□□□ □□□□□□□): ", round(phase_shift, digits=2
end
```

```
□□□□□□ □□□□□□□□□:
□□□□□□ □□□ □□□□□: □□□□□ = 33.7, □□□□□□□□ = 46.89
□□□□□□ □□□ □□□□□□□□: □□□□□ = 2.9, □□□□□□□□ = 10.41
□□□□□ □□□ (□□□□□□□ □□□□□□□): -30.8
```

15 Сохранение графиков

```
savefig(plt1, plotsdir(script_name, "lv_dynamics.png"))
savefig(plt2, plotsdir(script_name, "lv_phase_portrait.png"))
savefig(plt3, plotsdir(script_name, "lv_derivatives.png"))
savefig(plt4, plotsdir(script_name, "lv_relative_changes.png"))
savefig(plt5, plotsdir(script_name, "lv_spectrum.png"))
savefig(plt6, plotsdir(script_name, "lv_panel.png"))
```

```
┌ Warning: No strict ticks found
└ @ PlotUtils C:\Users\12232\.julia\packages\PlotUtils\HX80C\src\tic
┌ Warning: No strict ticks found
└ @ PlotUtils C:\Users\12232\.julia\packages\PlotUtils\HX80C\src\tic
┌ Warning: Invalid negative or zero value 0.0 found at series index 1 fo
└ @ Plots C:\Users\12232\.julia\packages\Plots\GIume\src\utils.jl:10
┌ Warning: Invalid negative or zero value 0.0 found at series index 1 fo
└ @ Plots C:\Users\12232\.julia\packages\Plots\GIume\src\utils.jl:10
┌ Warning: No strict ticks found
└ @ PlotUtils C:\Users\12232\.julia\packages\PlotUtils\HX80C\src\tic
┌ Warning: No strict ticks found
└ @ PlotUtils C:\Users\12232\.julia\packages\PlotUtils\HX80C\src\tic
┌ Warning: Invalid negative or zero value 0.0 found at series index 1 fo
└ @ Plots C:\Users\12232\.julia\packages\Plots\GIume\src\utils.jl:10
```

"C:\\Users\\12232\\Documents\\GitHub\\2026-1--study--simulation-modeling\\labs\\lab02\\project\\plots\\lv_panel.png"

16 Дополнительный анализ: чувствительность к параметрам

```
println("\n\n" * "="^60)
println("□□□□□□ □□□□□□□□□□□□□□□□")
println("="^60)
```

```
============================================================
□□□□□□ □□□□□□□□□□□□□□□□
============================================================
```

Функция для анализа влияния параметров

```
function analyze_parameter_sensitivity(param_index, values, param_na
    println("\n□□□□□□ □□□□□□□□□□□□□□□□ □ □□□□□□□□□: ", param_name)
    results = []

    for val in values
    p_test = copy(p_lv)
    p_test[param_index] = val
    prob_test = ODEProblem(lotka_volterra!, u0_lv, tspan_lv, p_test
```

```
    sol_test = solve(prob_test, dt = dt_lv)
    prey_end = sol_test.u[end][1]
    predator_end = sol_test.u[end][2]

    push!(results, (param=val, prey=prey_end, predator=predator_end
    println("  $(param_name)=$(val): □□□□□□=$(round(prey_end, digits=
    end
    return results
end
```

analyze_parameter_sensitivity (generic function with 1 method)

16.1 Анализ чувствительности к ключевым параметрам

```
if true  # □□□□□□□□□□ □ true □□□ □□□□□□□□□□ □□□□□□□
    println("\n1. □□□□□□□ □□□□□□□□ □□□□□□□□□□□ □□□□□ (□):")
    analyze_parameter_sensitivity(1, [0.05, 0.1, 0.2, 0.3], "□")

    println("\n2. □□□□□□□ □□□□□□□□□□ □□□□□□□□ (□):")
    analyze_parameter_sensitivity(4, [0.1, 0.3, 0.5, 0.7], "□")
end

println("\n□□□□□□□□□□□□□ □□□□□□□□□ □□□□□□□!")
```

1. □□□□□□□ □□□□□□□□ □□□□□□□□□□□ □□□□□ (□):

[illegible] [illegible] [illegible] [illegible]:

[illegible]=0.05: [illegible]=16.32, [illegible]=0.62

[illegible]=0.1: [illegible]=17.86, [illegible]=4.96

[illegible]=0.2: [illegible]=26.73, [illegible]=6.87

[illegible]=0.3: [illegible]=33.76, [illegible]=24.48

2. [illegible] [illegible] [illegible] ([illegible]):

[illegible] [illegible] [illegible]:

[illegible]=0.1: [illegible]=0.87, [illegible]=2.07

[illegible]=0.3: [illegible]=17.86, [illegible]=4.96

[illegible]=0.5: [illegible]=54.16, [illegible]=10.0

[illegible]=0.7: [illegible]=100.93, [illegible]=1.38

[illegible] [illegible] [illegible]!

17 Выводы

В ходе данной лабораторной работы я познакомился с основными моделями, такими как SIR и модель Лотки-Вольтерры. Я узнал где они применяются. Так же провел анализ чувствительности модели к параметрам.

Список литературы

::: {#Текст Лабораторной работы №2} :::